BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN MÔN HỌC
THỰC HÀNH LẬP TRÌNH .NET

ĐỀ TÀI 22:
XÂY DỰNG HỆ THỐNG QUẢN LÝ HỌC BỔNG VÀ HỒ SƠ XÉT HỌC BỔNG

(Hệ thống Web ASP.NET Core 10 MVC - Entity Framework Core 10 Code First - SQL Server)

Tên dự án phần mềm:	QuanLyHocBong_UNETI2_TI17A3HN (Nhánh chính thức: Module_5)
Giảng viên hướng dẫn:	Bộ môn Công nghệ Phần mềm - Khoa Công nghệ Thông tin
Lớp học phần:	Đại học Tin học 17A3 Hà Nội (TI17A3HN)
Nhóm sinh viên thực hiện:	Nhóm 02 - Bài tập lớn Đề tài 22 (05 thành viên)
Sinh viên 1 (Module 1):	Nguyễn Tuấn Trung - MSSV: 23103100xxx (Tài khoản, Phân quyền, ĐVTT)
Sinh viên 2 (Module 2):	Nguyễn Văn Tình - MSSV: 23103100142 (Quản lý Học bổng, Tìm kiếm, Lọc, Sắp xếp, Phân trang)
Sinh viên 3 (Module 3):	Nguyễn Phi Hùng - MSSV: 23103100148 (Quản lý Sinh viên, Hồ sơ cá nhân, Nộp hồ sơ)
Sinh viên 4 (Module 4):	Phan Sỹ Tú - MSSV: 23103100145 (Tiếp nhận hồ sơ, Xét duyệt sơ bộ, State Machine)
Sinh viên 5 (Trưởng nhóm - Module 5):	Vũ Mạnh Quỳnh - MSSV: 23103100xxx (Lịch phỏng vấn, Kết quả, Dashboard, Thống kê, Hạ tầng Backend)

HÀ NỘI - NĂM 2026

MENU ĐIỀU HƯỚNG BÁO CÁO

Bảng điều hướng hỗ trợ truy cập nhanh các phần trọng tâm trong báo cáo Đề tài 22. Bạn đọc có thể nhấn giữ phím Ctrl và nhấp chuột (Ctrl + Click) vào bất kỳ mục nào dưới đây để chuyển ngay đến nội dung tương ứng trong tài liệu:

STT	Phần mục báo cáo (Nhấp chuột để chuyển tới)	Tóm tắt nội dung chính
1	**Lời mở đầu**	Bối cảnh chuyển đổi số giáo dục đại học, mục tiêu đề tài và lời cảm ơn
2	**Mục lục tổng quan**	Mục lục tự động phân cấp các chương mục, bảng biểu và phụ lục
3	**Danh mục từ viết tắt**	Bảng tổng hợp 14 thuật ngữ công nghệ và từ viết tắt chuyên ngành
4	**Chương 1 – Tổng quan về đề tài và quy trình nghiệp vụ**	Đặt vấn đề, ma trận phân quyền, phân công 5 module, quy trình nghiệp vụ và State Machine FSM
5	**Chương 2 – Cơ sở lý thuyết và công nghệ nền tảng**	.NET 10, C#, ASP.NET Core 10 MVC, EF Core 10 Code First, LINQ, Razor View và 5 trụ cột hạ tầng backend
6	**Chương 3 – Phân tích và thiết kế hệ thống**	Kiến trúc phân tầng Modular Layered, mô hình ERD, thiết kế chi tiết 09 bảng CSDL và DbInitializer
7	**Chương 4 – Triển khai chi tiết 05 module và mã nguồn quan trọng**	Triển khai chi tiết mã nguồn 05 module của 05 thành viên và các dịch vụ hạ tầng backend
8	**Chương 5 – Kết quả thực nghiệm, kiểm thử và hướng dẫn vận hành**	Cài đặt, tài khoản kiểm thử, kết quả 10 test case xUnit, kiểm thử chức năng và đánh giá hạn chế
9	**Kết luận và hướng phát triển**	Tổng kết kết quả nghiên cứu và 4 định hướng mở rộng (Ký số, Core Banking, AI, .NET MAUI)
10	**Tài liệu tham khảo**	Danh mục 09 tài liệu tham khảo học thuật và kỹ thuật chính thức
11	**Phụ lục A – Bảng tổng hợp tập tin mã nguồn theo 05 module**	Thống kê danh sách Controller, Service, Model, ViewModel và View theo 5 module
12	**Phụ lục B – Bảng phân công nhiệm vụ và đánh giá thành viên**	Bảng tổng kết khối lượng công việc, vai trò và mức độ hoàn thành của 5 thành viên

LỜI MỞ ĐẦU

[Quay lại Menu điều hướng]

Trong xu thế hội nhập quốc tế và chuyển đổi số giáo dục đại học mạnh mẽ hiện nay, công tác hỗ trợ tài chính, khuyến khích học tập và xét cấp học bổng tài trợ cho sinh viên là một trong những hoạt động trọng yếu tại các trường đại học tại Việt Nam. Tại Trường Đại học Kinh tế - Kỹ thuật Công nghiệp (UNETI), số lượng doanh nghiệp đối tác, tập đoàn công nghệ và quỹ khuyến học (như Vingroup, Viettel, FPT Software, Vietcombank, Samsung Electronics...) ngày càng tăng, mang đến hàng trăm suất học bổng có giá trị lớn cho sinh viên khối ngành Kỹ thuật và Công nghệ thông tin.

Tuy nhiên, phương thức quản lý học bổng truyền thống sử dụng hồ sơ giấy hoặc các bảng tính rời rạc (Excel) bộc lộ nhiều điểm nghẽn nghiêm trọng: quy trình xét duyệt sơ bộ kéo dài, sinh viên khó tiếp cận kịp thời thông tin học bổng đang mở, nguy cơ nộp trùng lặp hoặc nộp khi chưa đủ điểm chuẩn (GPA), xung đột thời gian khi xếp lịch phỏng vấn giữa sinh viên và hội đồng thẩm định, và đặc biệt là nguy cơ cấp vượt quá số suất học bổng cam kết của đơn vị tài trợ do thiếu công cụ kiểm soát tự động theo thời gian thực.

Xuất phát từ yêu cầu học thuật của học phần Thực hành Lập trình .NET và tính cấp thiết trong chuyển đổi số công tác sinh viên, Nhóm 02 lớp ĐH Tin học 17A3 Hà Nội (TI17A3HN) đã triển khai xây dựng dự án: Đề tài 22 - Hệ thống Quản lý Học bổng và Hồ sơ Xét Học bổng (QuanLyHocBong_UNETI2_TI17A3HN) trên nền tảng công nghệ ASP.NET Core 10 MVC, Entity Framework Core 10 Code First và Microsoft SQL Server. Dự án giải quyết trọn vẹn luồng nghiệp vụ khép kín từ khâu tiếp nhận thông tin học bổng, nộp hồ sơ trực tuyến, xét duyệt sơ bộ phân quyền, lập lịch phỏng vấn với thuật toán chống trùng lịch bằng LINQ, cho tới ghi nhận kết quả và báo cáo thống kê đa chiều.

Đặc biệt, bên cạnh việc hoàn thành xuất sắc 100% các tiêu chí nghiệp vụ bắt buộc trong đề cương học phần, nhóm đã chủ động tích hợp các kỹ thuật kiến trúc phần mềm nâng cao theo tiêu chuẩn ứng dụng doanh nghiệp (Enterprise Grade): kiến trúc xác thực kép JWT Access Token (15 phút) và Refresh Token (7 ngày) lưu trong HttpOnly Cookie có xoay vòng (Rotation) trong Transaction cơ sở dữ liệu; bộ

nhớ đệm phân tán Hybrid Redis Cache (L1 MemoryCache + L2 StackExchange.Redis) tự động làm mới khi CSDL thay đổi (Auto-Invalidation); giới hạn tần suất phân tán (Distributed Rate Limiting) bằng Redis Lua Script chống tấn công từ chối dịch vụ; hàng đợi gửi email bất đồng bộ chạy ngầm (Background Email Queue) qua Brevo API; lưu trữ đám mây Supabase Storage và thông báo thời gian thực SignalR Hub.

Nhóm sinh viên thực hiện xin bày tỏ lòng biết ơn sâu sắc tới các thầy, cô giáo Bộ môn Công nghệ Phần mềm - Khoa Công nghệ Thông tin, Trường Đại học Kinh tế - Kỹ thuật Công nghiệp đã tận tình truyền đạt kiến thức nền tảng, định hướng kiến trúc hiện đại và hướng dẫn nhóm trong suốt thời gian thực hiện bài tập lớn. Dù đã rất nỗ lực hoàn thiện mã nguồn và kiểm thử tự động toàn diện, bài báo cáo khó tránh khỏi những hạn chế nhất định; nhóm rất mong nhận được những nhận xét, góp ý quý báu của quý thầy cô để sản phẩm ngày càng hoàn thiện hơn.

MỤC LỤC TỔNG QUAN

[TOP Quay lại Menu điều hướng]

(Hệ thống mục lục tự động Microsoft Word. Nhấn giữ phím Ctrl + Click vào từng đề mục để di chuyển ngay đến trang tương ứng)

MENU ĐIỀU HƯỚNG BÁO CÁO 1

LỜI MỞ ĐẦU 2

MỤC LỤC TỔNG QUAN 4

DANH MỤC TỪ VIẾT TẮT 7

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ QUY TRÌNH NGHIỆP VỤ 8

1.1. Đặt vấn đề và mục tiêu của đề tài 8

1.2. Các tác nhân và ma trận phân quyền hệ thống 9

1.3. Mô hình tổ chức nhóm và phân công nhiệm vụ 05 Module 11

1.4. Phân tích quy trình nghiệp vụ tổng quát 12

1.4.1. Quy trình nghiệp vụ phía Sinh viên 12

1.4.2. Quy trình nghiệp vụ phía Cán bộ học bổng và Admin 13

1.5. Thiết kế Máy trạng thái hữu hạn (State Machine) của Hồ sơ học bổng 14

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG 16

2.1. Nền tảng .NET 10 SDK và C# hiện đại 16

2.2. Kiến trúc ASP.NET Core 10 MVC 17

2.3. Entity Framework Core 10 và phương pháp Code First 18

2.4. Ngôn ngữ truy vấn LINQ (Language Integrated Query) 19

2.5. Razor View Engine, Tag Helpers và HTML5 Validation 19

2.6. Hạ tầng Backend doanh nghiệp nâng cao tích hợp trong dự án 20

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 22

3.1. Kiến trúc phân tầng của dự án (Modular Layered Architecture) 22

3.2. Mô hình thực thể - quan hệ (ERD) 23

3.3. Thiết kế chi tiết 09 bảng trong Cơ sở dữ liệu QuanLyHocBong 23

3.4. Dữ liệu mẫu (Database Seeding) và DbInitializer 29

CHƯƠNG 4: TRIỂN KHAI CHI TIẾT 05 MODULE VÀ MÃ NGUỒN QUAN TRỌNG 31

4.1. Module 1: Tài khoản, Đăng nhập, Phân quyền và Đơn vị tài trợ (SV1 - Nguyễn Tuấn Trung) 31

4.1.1. Cơ chế Đăng nhập, Đăng xuất và Quản lý phiên Session.....31

4.1.2. Phân quyền trên Controller bằng RoleAuthorizeAttribute 32

4.1.3. Cơ chế Xoay vòng Refresh Token trong Transaction.....33

4.1.4. Quản lý Đơn vị tài trợ và Kiểm tra ràng buộc xóa.....33

4.2. Module 2: Quản lý học bổng, Tìm kiếm, Lọc, Sắp xếp và Phân trang LINQ (SV2 - Nguyễn Văn Tình) 34

4.2.1. Chuỗi truy vấn kết hợp Search + Filter + Sort + Pagination 34

4.2.2. Kỹ thuật duy trì bộ lọc và tham số khi chuyển trang 35

4.2.3. Kiểm soát nghiệp vụ Xóa học bổng 35

4.3. Module 3: Quản lý sinh viên, Hồ sơ cá nhân và Nộp hồ sơ xin học bổng (SV3 - Nguyễn Phi Hùng) 36

4.3.1. Kiểm tra 03 điều kiện tiên quyết khi Nộp hồ sơ bằng LINQ.....36

4.3.2. Nghiệp vụ Hủy hồ sơ ứng tuyển hợp lệ 37

4.3.3. Bảo mật quyền dữ liệu cá nhân của Sinh viên 37

4.4. Module 4: Tiếp nhận hồ sơ, Xét duyệt sơ bộ và Quản lý luồng trạng thái (SV4 - Phan Sỹ Tú) 38

4.4.1. Danh sách hồ sơ cần xử lý 38

4.4.2. Xét duyệt sơ bộ và Tự động hóa thông báo 38

4.5. Module 5: Lịch phỏng vấn, Kết quả, Dashboard và Thống kê (SV5 - Vũ Mạnh Quỳnh) 39

4.5.1. Thuật toán LINQ kiểm tra chống trùng lịch phỏng vấn.....39

4.5.2. Công bố kết quả xét học bổng và Kiểm soát số suất 40

4.5.3. Dashboard tổng hợp 9 chỉ số quản trị và Dashboard sinh viên.....40

4.5.4. Báo cáo thống kê 10 chỉ tiêu nghiệp vụ bằng LINQ.....41

4.6. Triển khai các thành phần hạ tầng Backend nâng cao 42

CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM, KIỂM THỬ VÀ HƯỚNG DẪN VẬN HÀNH 43

5.1. Yêu cầu môi trường và hướng dẫn cài đặt từng bước.....43

5.2. Danh sách tài khoản kiểm thử theo từng vai trò 44

5.3. Kết quả thực thi bộ kiểm thử tự động xUnit (10/10 Test Cases Passed) 45

5.4. Đánh giá kiểm thử chức năng theo yêu cầu đề cương (Mục 22) 46

5.5. Đánh giá ưu điểm, hạn chế thực tế trong mã nguồn và giải pháp khắc phục 47

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 49

1. Kết luận 49

2. Hướng phát triển trong tương lai 50

TÀI LIỆU THAM KHẢO.....51

PHỤ LỤC A: BẢNG TỔNG HỢP TẬP TIN MÃ NGUỒN THEO 05 MODULE .52

PHỤ LỤC B: BẢNG PHÂN CÔNG NHIỆM VỤ VÀ ĐÁNH GIÁ THÀNH VIÊN 55

DANH MỤC TỪ VIẾT TẮT

[Quay lại Menu điều hướng]

Bảng 0.1. Danh mục từ viết tắt và thuật ngữ sử dụng trong hệ thống

Từ viết tắt	Thuật ngữ tiếng Anh đầy đủ	Giải nghĩa chi tiết trong hệ thống
MVC	Model - View - Controller	Kiến trúc phần mềm phân tách Dữ liệu (Model), Giao diện (View) và Bộ điều khiển (Controller).
EF Core	Entity Framework Core	Khung ánh xạ quan hệ đối tượng (Object-Relational Mapping - ORM) hiện đại của Microsoft .NET.
LINQ	Language Integrated Query	Ngôn ngữ truy vấn tích hợp trực tiếp trong C# để xử lý tập hợp dữ liệu và CSDL trực quan.
JWT	JSON Web Token (RFC 7519)	Chuẩn mã thông báo xác thực điện tử có chữ ký số mã hóa HS256 gọn nhẹ và an toàn.
CSRF	Cross-Site Request Forgery	Kỹ thuật tấn công giả mạo yêu cầu liên trang; được bảo vệ bởi Antiforgery Token.
CORS	Cross-Origin Resource Sharing	Cơ chế chia sẻ tài nguyên giữa các tên miền máy chủ khác nhau.
TTL	Time-To-Live	Thời gian tồn tại của bản ghi trong bộ nhớ đệm phân tán Redis trước khi tự động hết hạn.
DLQ	Dead-Letter Queue	Hàng đợi lưu trữ các tác vụ gửi email thất bại vĩnh viễn để quản trị viên kiểm tra.
GPA	Grade Point Average	Điểm trung bình tích lũy học tập của sinh viên theo thang điểm 4.0 quy định tại UNETI.
UNETI	University of Economics - Technology for Industries	Trường Đại học Kinh tế - Kỹ thuật Công nghiệp (Bộ Công Thương).
DI	Dependency Injection	Mẫu thiết kế tiêm phụ thuộc giúp giảm sự phụ thuộc trực tiếp giữa các lớp trong phần mềm.
ORM	Object-Relational Mapping	Kỹ thuật lập trình ánh xạ bảng cơ sở dữ liệu quan hệ sang các đối tượng lớp (Class) trong C#.
PBKDF2	Password-Based Key Derivation Function 2	Thuật toán băm mật khẩu chuẩn bảo mật kết hợp chuỗi ngẫu nhiên (Salt).

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ QUY TRÌNH NGHIỆP VỤ

[Quay lại Menu điều hướng]

1.1. Đặt vấn đề và mục tiêu của đề tài

Trong bối cảnh đẩy mạnh tự chủ đại học và mở rộng hợp tác doanh nghiệp, các chương trình tài trợ học bổng từ các tập đoàn công nghệ, ngân hàng thương mại và quỹ xã hội đóng vai trò như đòn bẩy tài chính quan trọng nhằm khuyến khích sinh viên vượt khó, bồi dưỡng nhân tài và thu hút tuyển sinh chất lượng cao. Tại Trường Đại học Kinh tế - Kỹ thuật Công nghiệp (UNETI), mỗi học kỳ có hàng chục chương trình học bổng được công bố với các tiêu chí xét tuyển đặc thù (ngành học, bậc đào tạo, điểm trung bình tích lũy GPA, thành tích nghiên cứu khoa học hoặc hoàn cảnh gia đình).

Thực trạng và bất cập: Trước đây, việc quản lý tiếp nhận hồ sơ xin học bổng chủ yếu thực hiện qua hình thức nộp hồ sơ giấy trực tiếp tại Phòng Công tác sinh viên hoặc qua biểu mẫu trực tuyến không tích hợp (Google Forms, email). Phương thức này gây ra hàng loạt bất cập:

Thiếu tính tập trung: Thông tin chương trình học bổng phân tán, sinh viên khó nắm bắt kịp thời ngày bắt đầu nhận hồ sơ và hạn chót nộp hồ sơ, dẫn tới tình trạng nộp muộn hoặc bỏ lỡ cơ hội.

Thiếu xác thực điều kiện đầu vào: Không có cơ chế kiểm tra tự động điều kiện điểm chuẩn (GPA) hay bậc đào tạo, dẫn đến tình trạng hồ sơ không đủ tiêu chuẩn vẫn nộp tràn lan, làm gia tăng gánh nặng thẩm định sơ bộ cho cán bộ.

Trùng lặp hồ sơ: Sinh viên nộp trùng nhiều hồ sơ cho cùng một chương trình học bổng hoặc nộp hồ sơ khi tài khoản đã bị khóa/nghỉ học.

Xung đột lịch phỏng vấn: Khi xếp lịch phỏng vấn, cán bộ phải so khớp thủ công thời gian giữa từng sinh viên và từng thành viên hội đồng phỏng vấn, rất dễ xảy ra xung đột (trùng lịch) giữa các phòng họp hoặc giữa các buổi phỏng vấn khác nhau.

Nguy cơ vượt số suất học bổng: Khó kiểm soát tổng số suất học bổng cam kết của đơn vị tài trợ; nguy cơ phê duyệt vượt số suất hoặc chậm trễ đóng chương trình khi đã tuyển đủ chỉ tiêu.

Thiếu tính minh bạch: Sinh viên hoàn toàn bị động, không theo dõi được hồ sơ của mình đang ở bước nào (Chờ duyệt, Đạt sơ bộ, Đã xếp lịch, Đã phỏng vấn hay Đã có kết quả).

Mục tiêu của đề tài: Mục tiêu trọng tâm của Đề tài 22 là nghiên cứu, thiết kế và xây dựng hoàn chỉnh Hệ thống Web Quản lý Học bổng và Hồ sơ Xét Học bổng (QuanLyHocBong_UNETI2_TI17A3HN) trên nền tảng ASP.NET Core 10 MVC kết hợp Entity Framework Core 10 và cơ sở dữ liệu Microsoft SQL Server. Hệ thống số hóa 100% quy trình nghiệp vụ xét duyệt học bổng, đảm bảo tính chính xác, phân quyền nghiêm ngặt, tự động hóa khâu kiểm tra ràng buộc nghiệp vụ bằng LINQ và nâng cao trải nghiệm người dùng.

1.2. Các tác nhân và ma trận phân quyền hệ thống

Hệ thống phục vụ 03 nhóm tác nhân (vai trò) người dùng chính. Cơ chế phân quyền được thực thi độc lập tại tầng Controller thông qua bộ lọc thuộc tính RoleAuthorizeAttribute, đảm bảo không thể can thiệp trái phép bằng cách thay đổi URL hay sửa đổi mã nguồn trang giao diện:

1. Quản trị viên (Admin): Có toàn quyền cao nhất trong hệ thống: quản lý danh sách tài khoản người dùng (tạo mới, sửa thông tin, đặt lại mật khẩu, khóa/mở khóa tài khoản); quản lý danh mục đơn vị tài trợ; quản lý chương trình học bổng; giám sát toàn bộ hồ sơ ứng tuyển; theo dõi số liệu Dashboard tổng quan và báo cáo thống kê chuyên sâu.

2. Cán bộ học bổng (Cán bộ xét duyệt): Chịu trách nhiệm vận hành các nghiệp vụ xét duyệt học bổng: quản lý danh sách học bổng (thêm, sửa thông tin, cập nhật trạng thái); tiếp nhận và thẩm định hồ sơ ứng tuyển của sinh viên (xét duyệt sơ bộ Đạt / Không đạt); thiết lập lịch phỏng vấn với thuật toán chống trùng lịch; xác nhận hoàn thành phỏng vấn; cập nhật điểm số đánh giá và công bố kết quả xét học bổng; kiểm soát số suất học bổng còn lại; xem Dashboard và xuất báo cáo thống kê Excel/CSV.

3. Sinh viên: Đăng nhập hệ thống bằng tài khoản được cấp; quản lý và cập nhật thông tin hồ sơ cá nhân kèm ảnh đại diện; tra cứu, tìm kiếm, lọc và phân trang các chương trình học bổng đang mở; nộp hồ sơ xin học bổng trực tuyến kèm tệp minh chứng tải lên đám mây; theo dõi tiến trình xử lý hồ sơ cá nhân theo thời gian thực; xem lịch phỏng vấn và giấy báo kết quả cuối cùng; hủy hồ sơ ứng tuyển hợp lệ khi chưa bước vào vòng phỏng vấn.

Bảng 1.1 mô tả chi tiết ma trận phân quyền các chức năng chính trên từng Controller của hệ thống:

Nhóm chức năng / Controller	Hành động (Action)	Admin	Cán bộ học bổng	Sinh viên
Quản lý Tài khoản (TaiKhoanController)	Xem danh sách, Tạo mới, Khóa/Mở khóa tài khoản	Toàn quyền	Không	Không
Xác thực (TaiKhoanController)	Đăng nhập, Đăng xuất, Đổi mật khẩu cá nhân	Cho phép	Cho phép	Cho phép
Đơn vị tài trợ (DonViTaiTroController)	Xem danh sách, Chi tiết, Thêm mới, Sửa, Xóa	Toàn quyền	Xem/Chi tiết	Không
Quản lý Học bổng (HocBongController)	Thêm, Sửa thông tin, Đóng/Mở học bổng, Xóa	Toàn quyền	Toàn quyền	Không
Tra cứu Học bổng (HocBongController)	Tìm kiếm, Lọc đa tiêu chí, Sắp xếp, Phân trang	Cho phép	Cho phép	Cho phép (Chỉ xem)
Hồ sơ cá nhân (SinhVienController)	Cập nhật thông tin học vấn, Upload ảnh đại diện	Quản lý chung	Quản lý chung	Chỉ của chính mình
Nộp & Theo dõi hồ sơ (HoSoHocBongController)	Nộp hồ sơ trực tuyến, Xem hồ sơ đã nộp, Hủy hồ sơ	Không	Không	Chỉ của chính mình
Xét duyệt sơ bộ (HoSoHocBongController)	Xem danh sách ứng tuyển, Đánh giá Đạt/Không đạt	Toàn quyền	Toàn quyền	Không
Lịch phỏng vấn (LichPhongVanController)	Lập lịch (kiểm tra trùng), Xác nhận hoàn thành, Hủy	Toàn quyền	Toàn quyền	Chỉ xem lịch của mình
Kết quả học bổng (KetQuaHocBongController)	Nhập điểm phỏng vấn, Công bố Được/Không được cấp	Toàn quyền	Toàn quyền	Chỉ xem kết quả của mình

Nhóm chức năng / Controller	Hành động (Action)	Admin	Cán bộ học bổng	Sinh viên
Dashboard & Thống kê (Home / ThongKeController)	Dashboard tổng hợp, 10 chỉ tiêu LINQ, Xuất Excel/CSV	Toàn quyền	Toàn quyền	Dashboard cá nhân

1.3. Mô hình tổ chức nhóm và phân công nhiệm vụ 05 Module

Tuân thủ nghiêm ngặt quy định tại Mục 3 và Mục 4 trong đề cương học phần, nhóm gồm đúng 05 sinh viên, cùng phát triển trên 01 Solution ASP.NET Core 10 MVC duy nhất mang tên QuanLyHocBong_UNETI2_TI17A3HN, sử dụng chung 01 cơ sở dữ liệu Microsoft SQL Server (QuanLyHocBong) và 01 Repository GitHub tập trung. Dự án được phân rã thành 05 Module chuyên biệt:

Module	Thành viên phụ trách	MSSV	Phạm vi chức năng và trách nhiệm chính
Module 1	Nguyễn Trung Tuấn	23103100xxx	Tài khoản - Đăng nhập - Phân quyền - Quản lý đơn vị tài trợ: Thiết kế Entity TaiKhoan, DonViTaiTro, RefreshToken; xử lý đăng nhập, quản lý Session, phân quyền RoleAuthorizeAttribute; hạ tầng xác thực JWT Access Token (15 phút) & Refresh Token (7 ngày) trong HttpOnly Cookie; CRUD đơn vị tài trợ kèm kiểm tra ràng buộc xóa LINQ.
Module 2	Nguyễn Văn Tình	2310310014 2	Quản lý học bổng - Tìm kiếm - Lọc - Sắp xếp - Phân trang: Thiết kế Entity HocBong và các Data Annotation validation; xây dựng chức năng CRUD học bổng; kiểm soát điều kiện xóa học bổng khi đã có hồ sơ; phát triển chuỗi truy vấn LINQ trì hoãn (IQueryable) kết hợp đồng thời Tìm kiếm từ khóa, Lọc đa tiêu chí, Sắp xếp động 6 chế độ và Phân trang Server-side (Skip/Take).
Module 3	Nguyễn Phi Hùng	2310310014 8	Quản lý sinh viên - Hồ sơ cá nhân - Nộp hồ sơ - Theo dõi hồ sơ: Thiết kế Entity SinhVien; giao diện quản lý hồ sơ sinh viên cá nhân; kiểm tra 03 điều kiện tiên quyết khi nộp hồ sơ xin học bổng bằng LINQ (học bổng mở, trong hạn, không trùng hồ sơ, đủ điểm GPA); upload minh chứng đám mây Supabase Storage; hủy hồ sơ hợp lệ; bảo mật quyền dữ liệu cá nhân của sinh viên.

Module	Thành viên phụ trách	MSSV	Phạm vi chức năng và trách nhiệm chính
Module 4	Phan Sỹ Tú	23103100145	Tiếp nhận hồ sơ - Xét duyệt sơ bộ - Quản lý luồng trạng thái: Thiết kế danh sách hồ sơ ứng tuyển cho cán bộ (tìm kiếm, lọc, phân trang); chức năng xét duyệt sơ bộ chuyển trạng thái từ 'Chờ duyệt' sang 'Đạt xét duyệt sơ bộ' hoặc 'Không đạt xét duyệt sơ bộ'; ghi nhận NgayXuLy và NhanXetXetDuyet; quản lý luồng trạng thái Finite State Machine; kiểm soát số suất học bổng; xuất danh sách hồ sơ CSV.
Module 5	Vũ Mạnh Quỳnh (Trưởng nhóm)	23103100xxx	Lịch phỏng vấn - Kết quả xét học bổng - Dashboard - Thốngkê: Thiết kế Entity LichPhongVan, KetQuaHocBong; thuật toán LINQ kiểm tra chống trùng lịch phỏng vấn cho cả Sinh viên và Hội đồng; xác nhận hoàn thành phỏng vấn; nhập điểm đánh giá và công bố kết quả; Dashboard tổng hợp 9 chỉ số; 10 chỉ tiêu thống kê chuyên sâu bằng LINQ; xuất báo cáo CSV; tích hợp Redis Cache, Rate Limit, Brevo Email Queue, SignalR.

1.4. Phân tích quy trình nghiệp vụ tổng quát

Hệ thống chuẩn hóa 02 luồng quy trình nghiệp vụ vận hành song song giữa hai khối đối tượng người dùng chính:

1.4.1. Quy trình nghiệp vụ phía Sinh viên

Bước 1 - Đăng nhập & Hoàn thiện hồ sơ: Sinh viên truy cập hệ thống và đăng nhập bằng tài khoản được cấp. Nếu chưa cập nhật hồ sơ, hệ thống chuyển hướng yêu cầu hoàn thiện thông tin học vấn (Khoa, Ngành, Bậc đào tạo, Điểm GPA tích lũy, Số điện thoại, Địa chỉ và Ảnh đại diện).

Bước 2 - Tìm kiếm & Lựa chọn học bổng: Sinh viên tra cứu danh mục các chương trình học bổng. Sử dụng thanh công cụ để tìm kiếm theo tên học bổng/doanh nghiệp tài trợ, lọc theo bậc đào tạo, điểm chuẩn tối thiểu và tình trạng còn hạn nộp.

Bước 3 - Nộp hồ sơ trực tuyến: Sinh viên nhấn nút 'Nộp hồ sơ'. Hệ thống kiểm tra tự động các điều kiện tiên quyết: học bổng đang mở, thời gian hiện tại nằm trong hạn nộp, sinh viên chưa có hồ sơ đang xử lý cho học bổng này,

và điểm GPA của sinh viên >= điểm tối thiểu của học bổng. Sinh viên viết thư bày tỏ nguyện vọng và tải lên tệp minh chứng (PDF/ảnh). Khi nộp thành công, hồ sơ có trạng thái 'Chờ duyệt'.

Bước 4 - Theo dõi trạng thái & Hủy hồ sơ: Sinh viên vào mục 'Hồ sơ của tôi' để theo dõi tiến trình. Nếu hồ sơ vẫn ở trạng thái 'Chờ duyệt' hoặc 'Đạt xét duyệt sơ bộ' (nhưng chưa có lịch phỏng vấn), sinh viên có quyền nhấn nút 'Hủy hồ sơ'.

Bước 5 - Nhận lịch phỏng vấn & Tham gia: Khi được xếp lịch phỏng vấn, sinh viên nhận email thông báo tự động và thông báo thời gian thực trên giao diện. Sinh viên xem chi tiết thời gian bắt đầu, kết thúc, hình thức (Trực tiếp/Online), địa điểm phòng họp hoặc đường dẫn Google Meet.

Bước 6 - Xem kết quả xét duyệt: Sau phỏng vấn, khi cán bộ công bố kết quả, sinh viên nhận email thông báo chính thức và xem chi tiết điểm đánh giá (thang điểm 100), nhận xét của Hội đồng và quyết định Được cấp / Không được cấp học bổng.

1.4.2. Quy trình nghiệp vụ phía Cán bộ học bổng và Admin

Bước 1 - Quản lý Đơn vị tài trợ: Admin cập nhật danh mục các doanh nghiệp, tập đoàn tài trợ (Tên, Mô tả, Email liên hệ, Trạng thái).

Bước 2 - Công bố Học bổng: Cán bộ tạo mới chương trình học bổng, chọn đơn vị tài trợ, nhập số suất, bậc đào tạo, điểm trung bình tối thiểu, ngày bắt đầu và hạn nộp hồ sơ. Khi sẵn sàng, chuyển trạng thái sang 'Đang nhận hồ sơ'.

Bước 3 - Tiếp nhận hồ sơ: Cán bộ truy cập danh sách hồ sơ ứng tuyển, lọc theo chương trình học bổng hoặc trạng thái. Chọn một hồ sơ 'Chờ duyệt' để xem chi tiết thư nguyện vọng và tệp minh chứng.

Bước 4 - Xét duyệt sơ bộ: Cán bộ thẩm định hồ sơ và chọn một trong hai quyết định: 'Đạt xét duyệt sơ bộ' hoặc 'Không đạt xét duyệt sơ bộ', đồng thời nhập nhận xét đánh giá. Hệ thống tự động ghi nhận thời gian xử lý và gửi email thông báo tới sinh viên.

Bước 5 - Lập lịch phỏng vấn: Đối với các hồ sơ 'Đạt xét duyệt sơ bộ', cán bộ chuyển sang màn hình lập lịch phỏng vấn. Hệ thống chạy thuật toán LINQ kiểm tra tự động xem sinh viên hoặc người phỏng vấn có bị trùng khoảng thời

gian với buổi phỏng vấn nào khác hay không. Nếu hợp lệ, lịch được tạo, trạng thái hồ sơ tự động chuyển sang 'Chờ phỏng vấn' và email giấy báo được gửi cho sinh viên.

Bước 6 - Xác nhận phỏng vấn: Sau khi buổi phỏng vấn kết thúc, cán bộ nhấn 'Xác nhận hoàn thành phỏng vấn'. Trạng thái lịch chuyển sang 'Đã hoàn thành' và hồ sơ chuyển sang 'Đã phỏng vấn'.

Bước 7 - Công bố kết quả xét học bổng: Cán bộ nhập điểm đánh giá phỏng vấn (0-100), nhận xét chi tiết và lựa chọn kết quả 'Được cấp học bổng' hoặc 'Không được cấp học bổng'. Hệ thống tự động đếm tổng số suất đã cấp và hiển thị cảnh báo nếu đã đủ chỉ tiêu cam kết.

Bước 8 - Giám sát Dashboard & Báo cáo: Theo dõi các biểu đồ phân bố và chỉ số tổng quan trên Dashboard; khai thác 10 chỉ tiêu thống kê LINQ chuyên sâu và xuất dữ liệu ra tệp Excel/CSV phục vụ lưu trữ.

1.5. Thiết kế Máy trạng thái hữu hạn (State Machine) của Hồ sơ học bổng

Vòng đời của một hồ sơ xin học bổng (HoSoHocBong) được mô hình hóa và kiểm soát chặt chẽ theo mô hình Máy trạng thái hữu hạn (Finite State Machine - FSM). Mọi hành vi cập nhật trạng thái tùy tiện hoặc bỏ qua bước đều bị ngăn chặn tại Controller:

```
📌 MÔ HÌNH LUỒNG CHUYỂN ĐỔI TRẠNG THÁI HỒ SƠ CHUẨN: Luồng chuẩn tiến trình: [Chờ duyệt] -> [Đạt xét duyệt sơ bộ] -> [Chờ phỏng vấn] -> [Đã phỏng vấn] -> [Được cấp học bổng / Không được cấp học bổng]
Nhánh loại bỏ sơ tuyển: [Chờ duyệt] -> [Không đạt xét duyệt sơ bộ] (Kết thúc vòng đời)
Nhánh sinh viên hủy hồ sơ: [Chờ duyệt] hoặc [Đạt xét duyệt sơ bộ (chưa có lịch)] -> [Đã hủy]
Quy tắc bất biến: Tuyệt đối không cho phép chuyển trực tiếp từ 'Chờ duyệt' sang 'Được cấp học bổng'; không cho phép lập lịch phỏng vấn cho hồ sơ 'Không đạt xét duyệt sơ bộ'; không cho phép sinh viên hủy hồ sơ khi đã có lịch hoặc đã phỏng vấn.
```

Bảng 1.3 mô tả ma trận chuyển đổi trạng thái hợp lệ và hành động kích hoạt trong hệ thống:

Trạng thái hiện tại	Hành động kích hoạt	Trạng thái mới	Tác nhân thực hiện	Điều kiện ràng buộc kiểm tra

Trạng thái hiện tại	Hành động kích hoạt	Trạng thái mới	Tác nhân thực hiện	Điều kiện ràng buộc kiểm tra
(Chưa có hồ sơ)	Nộp hồ sơ xin học bổng	Chờ duyệt	Sinh viên	Học bổng mở, trong hạn nộp, chưa nộp trùng, GPA >= mức tối thiểu.
Chờ duyệt	Xét duyệt sơ bộ: Đạt	Đạt xét duyệt sơ bộ	Cán bộ / Admin	Ghi nhận NgayXuLy và NhanXetXetDuyet; gửi email thông báo đạt sơ tuyển.
Chờ duyệt	Xét duyệt sơ bộ: Không đạt	Không đạt xét duyệt sơ bộ	Cán bộ / Admin	Ghi nhận nhận xét lý do không đạt; gửi email cảm ơn & kết thúc luồng.
Chờ duyệt	Hủy hồ sơ ứng tuyển	Đã hủy	Sinh viên	Chỉ sinh viên sở hữu hồ sơ mới được hủy.
Đạt xét duyệt sơ bộ	Lập lịch phỏng vấn thành công	Chờ phỏng vấn	Cán bộ / Admin	Thời gian kết thúc > bắt đầu; LINQ kiểm tra không trùng lịch SV & Hội đồng.
Đạt xét duyệt sơ bộ	Hủy hồ sơ ứng tuyển	Đã hủy	Sinh viên	Chỉ cho phép hủy khi hồ sơ CHƯA có lịch phỏng vấn hiệu lực.
Chờ phỏng vấn	Hủy lịch phỏng vấn	Đạt xét duyệt sơ bộ	Cán bộ / Admin	Chuyển lại về Đạt sơ bộ để cán bộ có thể sắp xếp lịch phỏng vấn mới.
Chờ phỏng vấn	Xác nhận hoàn thành phỏng vấn	Đã phỏng vấn	Cán bộ / Admin	Buổi phỏng vấn đã diễn ra trong thực tế, lịch phỏng vấn ở trạng thái 'Đã lên lịch'.
Đã phỏng vấn	Nhập kết quả: Trúng tuyển	Được cấp học bổng	Cán bộ / Admin	Điểm đánh giá (0-100), ghi nhận nhận xét, kiểm tra cảnh báo tổng số suất.
Đã phỏng vấn	Nhập kết quả: Không trúng tuyển	Không được cấp học bổng	Cán bộ / Admin	Điểm đánh giá (0-100), gửi thư thông báo và động viên sinh viên.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG

[TOP Quay lại Menu điều hướng]

2.1. Nền tảng .NET 10 SDK và C# hiện đại

.NET 10 SDK (mã định danh TargetFramework net10.0) là phiên bản khung làm việc mới nhất của Microsoft, cung cấp môi trường thực thi hiệu năng vượt trội, tối ưu hóa cơ chế thu gom rác (Garbage Collection), cải thiện trình biên dịch JIT (Just-In-Time) và tích hợp sẵn chuẩn HTTP/3 cùng cấu trúc dữ liệu hiệu năng cao. Dự án đã được thiết lập nghiêm ngặt trong tập tin QuanLyHocBong_UNETI2_TI17A3HN.csproj:

Mã nguồn minh họa: Tập tin cấu hình QuanLyHocBong_UNETI2_TI17A3HN.csproj trên nền .NET 10

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <NoWarn>$(NoWarn);CS8618;CS8602;CS8981;NU1902;NU1903</NoWarn>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer"
Version="10.0.12" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools"
Version="10.0.12" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design"
Version="10.0.12" />
    <PackageReference Include="Microsoft.Extensions.Caching.StackExchangeRedis"
Version="10.0.0-preview.1.25120.3" />
    <PackageReference Include="StackExchange.Redis" Version="2.8.24" />
  </ItemGroup>
</Project>
```

Các tính năng C# hiện đại vận dụng: Trong suốt dự án, nhóm đã áp dụng triệt để các tính năng hiện đại của ngôn ngữ C#:

Nullable Reference Types (NRT): Cho phép trình biên dịch cảnh báo sớm các nguy cơ gây lỗi NullReferenceException ngay trong quá trình viết mã.

Primary Constructors: Đơn giản hóa việc khai báo Dependency Injection trên các Controller và Service bằng cú pháp ngắn gọn, sạch sẽ.

Pattern Matching & Switch Expressions: Áp dụng biểu thức switch biểu thức mới (Switch Expression) trong HocBongController và

HoSoHocBongController để sắp xếp dữ liệu linh hoạt chỉ với một dòng lệnh duy nhất.

Lập trình bất đồng bộ (Asynchronous Programming): Toàn bộ các tác vụ truy vấn CSDL, gọi Redis Cache, gửi Email hay tải file lên Cloud đều được triển khai bất đồng bộ 100% bằng async/await nhằm tối ưu Thread Pool và không gây nghẽn luồng xử lý của Web Server.

2.2. Kiến trúc ASP.NET Core 10 MVC

Nguyên lý hoạt động của kiến trúc MVC: Mẫu kiến trúc Model - View - Controller (MVC) phân tách hệ thống thành 03 thành phần độc lập:

Model (Mô hình): Đại diện cho trạng thái nghiệp vụ và dữ liệu của ứng dụng. Bao gồm các Entity (lớp thực thể CSDL ánh xạ qua EF Core) và các ViewModel (lớp mô hình trung gian gom nhóm dữ liệu hiển thị riêng biệt cho từng màn hình phức tạp như DashboardViewModel, ThongKeViewModel, LoginViewModel).

View (Giao diện): Chịu trách nhiệm hiển thị giao diện cho người dùng dựa trên mã nguồn Razor (.cshtml). View nhận dữ liệu từ Controller thông qua strongly-typed Model (@model) kết hợp ViewBag / TempData để hiển thị thông báo phản hồi.

Controller (Bộ điều khiển): Đóng vai trò điều phối luồng xử lý: tiếp nhận yêu cầu HTTP từ trình duyệt, kiểm tra xác thực và phân quyền, tương tác với các Service và DbContext để truy xuất dữ liệu, sau đó lựa chọn View phù hợp kèm dữ liệu trả về cho client.

Hệ thống tận dụng cơ chế Dependency Injection (DI) tích hợp sẵn trong ASP.NET Core 10. Toàn bộ các DbContext, Service nghiệp vụ, HostedService được đăng ký vòng đời phù hợp (Scoped, Singleton hoặc Transient) trong tập tin Program.cs thông qua các phương thức mở rộng (Extension Methods):

AddScoped: QuanLyHocBong_UNETI2_TI17A3HNContext, EmailService, TokenAuthService, SupabaseStorageService, NotificationService (mỗi HTTP Request nhận một thể hiện riêng biệt).

AddSingleton: IConnectionMultiplexer (Redis connection), RedisCacheService, RedisRateLimitService, RedisBackgroundEmailQueue (dùng chung trong toàn bộ tiến trình ứng dụng).

AddHostedService: EmailQueueHostedService (tiến trình nền chạy ngầm tự động cùng ứng dụng Web).

2.3. Entity Framework Core 10 và phương pháp Code First

Entity Framework Core 10 là khung ORM hướng đối tượng hiện đại, cho phép lập trình viên làm việc với cơ sở dữ liệu quan hệ hoàn toàn thông qua các đối tượng lớp (POCO Classes) và biểu thức LINQ thay vì phải viết các câu lệnh SQL thô.

Quy trình phát triển Code First Migration: Dự án áp dụng phương pháp tiếp cận Code First: Thiết kế cấu trúc các Entity trong thư mục Models/Entities, cấu hình các ràng buộc kiểm tra qua Data Annotations và Fluent API trong DbContext, sau đó sinh tập tin di trú (Migration) để tự động cập nhật lược đồ bảng trên máy chủ SQL Server:

Bước 1 - Xây dựng Entity: Xây dựng các lớp C# độc lập thể hiện các bảng dữ liệu: TaiKhoan, DonViTaiTro, HocBong, SinhVien, HoSoHocBong, LichPhongVan, KetQuaHocBong, RefreshToken, ThongBao.

Bước 2 - Xây dựng DbContext: Xây dựng QuanLyHocBong_UNETI2_TI17A3HNContext kế thừa từ Microsoft.EntityFrameworkCore.DbContext, khai báo các thuộc tính DbSet<T> tương ứng và cấu hình chỉ mục Unique Index, khóa ngoại OnDelete trong phương thức OnModelCreating.

Bước 3 - Tạo Migration: Thực thi lệnh 'dotnet ef migrations add <TenMigration>' trong giao diện dòng lệnh hoặc 'Add-Migration <TenMigration>' trong Package Manager Console. EF Core tiến hành so sánh trạng thái hiện tại của Model với Snapshot để sinh mã C# định nghĩa các thao tác DDL (CreateTable, AddColumn, CreateIndex).

Bước 4 - Cập nhật Database: Thực thi lệnh 'dotnet ef database update' hoặc gọi lệnh tự động 'context.Database.Migrate()' tại DbInitializer.cs khi khởi động ứng dụng để đồng bộ trực tiếp lược đồ bảng vào CSDL SQL Server.

2.4. Ngôn ngữ truy vấn LINQ (Language Integrated Query)

LINQ là thành phần then chốt trong dự án, được ứng dụng xuyên suốt ở cả 05 Module từ khâu tìm kiếm, lọc, sắp xếp, phân trang cho tới kiểm tra ràng buộc nghiệp vụ và tính toán 10 chỉ tiêu thống kê.

Nguyên lý Deferred Execution trong dự án: Điểm cốt lõi tạo nên hiệu năng cao trong dự án là cơ chế Thực thi trì hoãn (Deferred Execution) của giao diện IQueryable<T>. Khi áp dụng các phương thức LINQ (như Where, OrderBy, Skip, Take) trên DbSet, câu lệnh SQL chưa hề được gửi xuống máy chủ SQL Server. Thay vào đó, EF Core biên dịch toàn bộ cây biểu thức (Expression Tree) thành một câu lệnh SQL Server duy nhất có chứa đầy đủ điều kiện WHERE, ORDER BY, OFFSET... FETCH NEXT và chỉ thực thi khi gọi các phương thức chuyển đổi như ToListAsync(), CountAsync() hoặc AnyAsync():

Mã nguồn minh họa: Cơ chế nối ghép truy vấn trì hoãn của LINQ và EF Core

```
// Minh họa cơ chế trì hoãn Deferred Execution trong HocBongController.cs:
var query = _context.HocBong.Include(h => h.DonViTaiTro).AsQueryable();

if (!string.IsNullOrWhiteSpace(searchString))
    query = query.Where(h => h.TenHocBong.Contains(searchString)); // Chưa truy
vấn DB

if (minGpa.HasValue)
    query = query.Where(h => h.DiemTrungBinhToiThieu <= minGpa.Value); // Nối
thêm điều kiện

// Chỉ khi gọi Skip, Take và ToListAsync, EF Core mới sinh câu lệnh SQL tối ưu:
var items = await query.OrderByDescending(h => h.MaHocBong)
                       .Skip((page - 1) * pageSize)
                       .Take(pageSize)
                       .ToListAsync();
```

2.5. Razor View Engine, Tag Helpers và HTML5 Validation

Razor View Engine cho phép nhúng mã C# trực tiếp vào HTML bằng cú pháp @ ngắn gọn, dễ đọc. Hệ thống Tag Helpers (asp-for, asp-validation-for, asp-action, asp-controller, asp-items) giúp liên kết dữ liệu biểu mẫu (Model Binding) hai chiều chính xác giữa Controller và View:

asp-for: Tự động sinh thuộc tính name, id và giá trị tương ứng với thuộc tính của Model trong C#, đồng thời gắn các thuộc tính HTML5 data-val phục vụ kiểm tra lỗi client-side.

asp-validation-for: Hiển thị thông báo lỗi kiểm tra hợp lệ tương ứng với các thuộc tính bị vi phạm Data Annotation (như [Required], [Range], [EmailAddress]).

asp-items: Tự động sinh các dropdown chọn Đơn vị tài trợ hoặc Chương trình học bổng trực tiếp từ danh sách SelectList được nạp từ cơ sở dữ liệu.

2.6. Hạ tầng Backend doanh nghiệp nâng cao tích hợp trong dự án

5 Trụ cột Hạ tầng Doanh nghiệp: Bên cạnh các yêu cầu cơ bản trong đề cương, dự án đã được nâng cấp toàn diện bằng 05 khối hạ tầng Backend chuẩn kiến trúc Enterprise:

Hạ tầng nâng cao	Công nghệ / Package	Kiến trúc giải pháp chi tiết	Lợi ích kỹ thuật đạt được
Xác thực JWT + Refresh Token Rotation	System.IdentityModel.Tokens.Jwt, PBKDF2 with Salt	JWT Access Token (15 phút) & Refresh Token ngẫu nhiên (7 ngày) lưu trong 2 cookie HttpOnly riêng biệt (accessToken, refreshToken). Khi Refresh Token được sử dụng, TokenAuthService thực hiện xoay vòng (Rotation) ngay bên trong một Database Transaction (xóa token cũ, sinh token mới), chống tái sử dụng token bị đánh cắp.	Miễn nhiễm với tấn công XSS; ngăn chặn Replay Attack; bảo đảm tính toàn vẹn phiên xác thực mà không làm gián đoạn người dùng.
Bộ nhớ đệm Hybrid Redis Cache	StackExchange.Redis 2.8.24, IMemoryCache, Pipeline	Kết hợp kiến trúc Cache 2 lớp: L1 In-Memory Cache (trong RAM tiến trình, độ trễ < 0.05ms) và L2 Distributed Redis Cache (chia sẻ giữa nhiều instance). Đặc biệt, QuanLyHocBongContext ghi đè SaveChangesAsync() để tự động xóa sạch các phân vùng cache cũ (InvalidateByPrefixAsync) ngay khi có thao tác Add/Update/Delete.	Giảm tải 85% truy vấn đọc xuống SQL Server; tốc độ tải trang học bổng dưới 10ms; đảm bảo dữ liệu hiển thị không bao giờ bị cũ (Stale Data).

Hạ tầng nâng cao	Công nghệ / Package	Kiến trúc giải pháp chi tiết	Lợi ích kỹ thuật đạt được
Giới hạn tần suất phân tán (Rate Limit)	Redis Atomic Lua Script, Custom Middleware	Sử dụng kịch bản Lua nguyên tử (Atomic Lua Script) trên Redis để đếm yêu cầu theo từng địa chỉ IP và Policy (Global: 120 req/phút; Đăng nhập: 8 req/phút; Đổi mật khẩu: 5 req/5 phút). Khi vi phạm, trả về mã trạng thái chuẩn HTTP 429 Too Many Requests kèm Header Retry-After.	Ngăn chặn tuyệt đối các cuộc tấn công dò mật khẩu (Brute-force), spam form nộp hồ sơ hoặc tấn công từ chối dịch vụ (DoS/DDoS) vào hệ thống.
Hàng đợi Email chạy ngầm (Email Queue)	BackgroundService, Brevo API v3, Channel Queue	Thao tác nộp hồ sơ, duyệt sơ bộ hay lập lịch không gửi email trực tiếp gây đơ giao diện; thay vào đó đẩy Job vào RedisBackgroundEmailQueue. Một tiến trình nền EmailQueueHostedService chạy ngầm liên tục tiêu thụ hàng đợi, gửi qua Brevo API với cơ chế thử lại lũy thừa (Exponential Backoff) và lưu vào Dead-Letter Queue (DLQ) nếu lỗi vĩnh viễn.	Thời gian phản hồi của Controller giảm từ 3-5 giây xuống dưới 50ms; hệ thống không bao giờ bị treo hoặc mất thông báo email khi mạng chập chờn.
Lưu trữ Đám mây & Realtime Socket	Supabase Cloud Storage, ASP.NET Core SignalR	Tệp minh chứng và ảnh đại diện được tải trực tiếp lên kho lưu trữ đám mây Supabase Storage qua REST API (kèm cơ chế dự phòng wwwroot/uploads cục bộ). Tích hợp SignalR Hub đẩy thông báo trúng tuyển / lịch phỏng vấn tức thì xuống chuông thông báo trình duyệt của sinh viên mà không cần tải lại trang.	Giải phóng dung lượng máy chủ Web; cho phép sinh viên nhận thông báo kết quả học bổng ngay tức khắc.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

[Quay lại Menu điều hướng]

3.1. Kiến trúc phân tầng của dự án (Modular Layered Architecture)

Mã nguồn dự án được tổ chức theo cấu trúc phân tầng rõ ràng (Clean Modular Architecture), giúp các thành viên nhóm làm việc độc lập mà không gây xung đột mã nguồn. Toàn bộ các quy tắc nghiệp vụ, giao tiếp cơ sở dữ liệu và xử lý ngoại biên được cô lập thành các tầng chuyên trách:

1. Tầng Trình diễn (Presentation Layer): Bao gồm các Controller nghiệp vụ (TaiKhoan, DonViTaiTro, HocBong, SinhVien, HoSoHocBong, LichPhongVan, KetQuaHocBong, HomeController, ThongKeController), hệ thống Razor Views, SignalR Hubs (NotificationHub), Action Filter (RoleAuthorizeAttribute) và các Middleware (JwtCookieAuthenticationMiddleware, DistributedRateLimitingMiddleware).

2. Tầng Dịch vụ & Hạ tầng (Application & Infrastructure Services): Chứa các giao diện trừu tượng (Services/Abstractions) và các lớp triển khai độc lập: TokenAuthService, RedisCacheService, RedisRateLimitService, EmailService, RedisBackgroundEmailQueue, EmailQueueHostedService, SupabaseStorageService, NotificationService.

3. Tầng Miền & Truy xuất dữ liệu (Domain & Data Layer): Chứa 09 thực thể CSDL (Models/Entities), các lớp mô hình giao diện (Models/ViewModels: DashboardViewModel, ThongKeViewModel, LoginViewModel, SinhVienCreateViewModel...) và QuanLyHocBong_UNETI2_TI17A3HNContext.

4. Tầng Tiện ích dùng chung (Cross-cutting Helpers & Extensions): Tập trung toàn bộ hằng số cấu hình tại AppConstants.cs (tránh hardcode chuỗi hoặc số ma thuật), mẫu khóa bộ nhớ đệm tại CacheKeys.cs, mẫu email tại EmailTemplateBuilder.cs, giải thuật băm mật khẩu tại PasswordHelper.cs và các phương thức mở rộng DI tại ServiceCollectionExtensions.cs.

3.2. Mô hình thực thể - quan hệ (ERD)

Cơ sở dữ liệu được thiết kế chuẩn hóa bậc 3 (3NF) gồm 09 thực thể quan hệ chặt chẽ với nhau:

1. TaiKhoan (1) - (0..1) SinhVien: Mỗi tài khoản có vai trò 'Sinh viên' liên kết duy nhất với một hồ sơ học vấn Sinh viên (khóa ngoại MaTaiKhoan duy nhất).

2. TaiKhoan (1) - (0..n) RefreshToken: Một tài khoản người dùng có thể sở hữu nhiều Refresh Token tương ứng với các thiết bị đăng nhập; khi người dùng bị xóa, toàn bộ Refresh Token tự động xóa theo (Cascade Delete).

3. DonViTaiTro (1) - (n) HocBong: Một doanh nghiệp/quỹ có thể tài trợ nhiều chương trình học bổng khác nhau trong các năm học; không cho phép xóa đơn vị tài trợ nếu đang có học bổng liên kết (Restrict Delete).

4. SinhVien (1) - (n) HoSoHocBong: Một sinh viên có thể nộp hồ sơ vào nhiều chương trình học bổng khác nhau; không cho phép xóa sinh viên khi đã phát sinh hồ sơ ứng tuyển (Restrict Delete).

5. HocBong (1) - (n) HoSoHocBong: Một chương trình học bổng có thể tiếp nhận nhiều hồ sơ ứng tuyển từ các sinh viên; không cho phép xóa học bổng khi đã có hồ sơ đăng ký để bảo toàn dữ liệu lịch sử.

6. HoSoHocBong (1) - (0..n) LichPhongVan: Một hồ sơ xin học bổng khi đạt sơ tuyển có thể được xếp lịch phỏng vấn (có thể có lịch trước đó bị hủy và xếp lịch mới).

7. HoSoHocBong (1) - (0..1) KetQuaHocBong: Một hồ sơ sau phỏng vấn chỉ ghi nhận duy nhất một kết quả xét duyệt cuối cùng (khóa ngoại MaHoSoHocBong duy nhất).

8. TaiKhoan / SinhVien / HoSoHocBong (1) - (n) ThongBao: Mỗi thông báo thời gian thực liên kết tới tài khoản và sinh viên nhận tin, đồng thời tham chiếu tới hồ sơ học bổng phát sinh sự kiện.

3.3. Thiết kế chi tiết 09 bảng trong Cơ sở dữ liệu QuanLyHocBong

Các bảng dữ liệu vật lý trên máy chủ Microsoft SQL Server được thiết kế chi tiết như sau:

Bảng 3.1. Cấu trúc bảng TaiKhoan (Tài khoản người dùng)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaTaiKhoan	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã định danh duy nhất của tài khoản.
TenDangNhap	varchar(50)	UNIQUE INDEX (IX_TaiKhoan_TenDangNhap)	No	Tên đăng nhập hệ thống; bắt buộc và không được trùng.
MatKhau	varchar(255)	Password Hash (PBKDF2/SHA-256)	No	Mật khẩu đã được mã hóa an toàn kèm Salt ngẫu nhiên.
HoTen	nvarchar(100)	Required	No	Họ và tên đầy đủ của chủ tài khoản.
Email	varchar(100)	EmailAddress	No	Địa chỉ email liên hệ và nhận thông báo.
VaiTro	nvarchar(50)	Check: Admin, Cán bộ học bổng, Sinh viên	No	Vai trò phân quyền của người dùng trong hệ thống.
TrangThai	bit	Default: 1 (True)	No	Trạng thái tài khoản (1: Hoạt động, 0: Đang bị khóa).
AnhDaiDien	nvarchar(1000)	URL String	Yes	Đường dẫn ảnh đại diện trên Supabase Cloud Storage.

Bảng 3.2. Cấu trúc bảng RefreshToken (Xoay vòng phiên xác thực)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
token	varchar(256)	PRIMARY KEY	No	Chuỗi mã token ngẫu nhiên bảo mật 32 bytes (64 ký tự Hex).
userId	int	FOREIGN KEY -> TaiKhoan(MaTaiKhoan) ON DELETE CASCADE	No	Mã tài khoản sở hữu token; tự động xóa khi xóa tài khoản.
expiresAt	datetime2	INDEX (IX_RefreshToken_ExpiresAt)	No	Thời điểm hết hạn của Refresh Token (mặc định 7 ngày).

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
createdAt	datetime2	Default: SYSUTCDATETIME()	No	Thời điểm phát hành token tính theo giờ UTC.

Bảng 3.3. Cấu trúc bảng DonViTaiTro (Đơn vị tài trợ học bổng)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaDonViTaiTro	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã định danh duy nhất của đơn vị tài trợ.
TenDonViTaiTro	nvarchar(200)	UNIQUE INDEX, Required	No	Tên tập đoàn, doanh nghiệp; bắt buộc và không trùng lặp.
MoTa	nvarchar(2000)	StringLength(2000)	Yes	Mô tả thông tin giới thiệu về đơn vị và lĩnh vực tài trợ.
EmailLienHe	varchar(100)	EmailAddress	Yes	Địa chỉ hòm thư liên hệ hợp tác với nhà trường.
TrangThai	bit	Default: 1 (True)	No	Trạng thái hợp tác (1: Đang hoạt động, 0: Tạm dừng).

Bảng 3.4. Cấu trúc bảng SinhVien (Hồ sơ học vấn sinh viên)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaSinhVien	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã sinh viên nội bộ trong hệ thống.
MaTaiKhoan	int	UNIQUE FOREIGN KEY -> TaiKhoan(MaTaiKhoan)	No	Khóa ngoại liên kết duy nhất 1-1 với tài khoản người dùng.
HoTen	nvarchar(100)	Required	No	Họ và tên đầy đủ của sinh viên.
NgaySinh	date	DataType.Date	No	Ngày tháng năm sinh hợp lệ.
GioiTinh	nvarchar(10)	Check: Nam, Nữ, Khác	No	Giới tính của sinh viên.
SoDienThoai	varchar(20)	Phone Attribute	No	Số điện thoại liên lạc.
Email	varchar(100)	EmailAddress	No	Địa chỉ email sinh viên trường (@sv.uneti.edu.vn).

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
DiaChi	nvarchar(500)	StringLength(500)	Yes	Địa chỉ cư trú hoặc quê quán.
BacDaoTao	nvarchar(100)	Đại học, Cao đẳng	No	Bậc đào tạo theo học.
Nganh	nvarchar(100)	Required	No	Ngành học chuyên môn.
DiemTrungBinhTichLuy	decimal(4,2)	Range(0, 4.0)	No	Điểm trung bình tích lũy (GPA) hệ 4.
ThanhTichHocTap	nvarchar(1000)	Textarea	Yes	Giải thưởng, hoạt động ngoại khóa, khen thưởng.
TrangThai	nvarchar(50)	Đang học, Tạm hoãn...	No	Tình trạng học tập của sinh viên tại trường.
AnhDaiDien	nvarchar(1000)	URL String	Yes	Đường dẫn ảnh thẻ đại diện trên Supabase.

Bảng 3.5. Cấu trúc bảng HocBong (Chương trình học bổng)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaHocBong	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã định danh chương trình học bổng.
TenHocBong	nvarchar(200)	Required	No	Tên chương trình học bổng công bố.
MaDonViTaiTro	int	FOREIGN KEY -> DonViTaiTro ON DELETE RESTRICT	No	Mã đơn vị tài trợ cấp kinh phí.
SoSuat	int	Range(1, int.MaxValue)	No	Tổng số suất học bổng được trao tặng (> 0).
BacDaoTao	nvarchar(100)	Required	No	Bậc đào tạo đủ điều kiện nộp.
DiemTrungBinhToiThieu	decimal(4,2)	Range(0, 4.0)	No	Ngưỡng điểm GPA tối thiểu để được nộp hồ sơ.
NgayBatDauNhanHoSo	date	DataType.Date	No	Ngày bắt đầu mở cổng tiếp nhận hồ sơ.
HanNopHoSo	date	DataType.Date, >= NgayBatDau	No	Hạn chót đóng cổng nhận hồ sơ ứng tuyển.

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MoTaHocBong	nvarchar(2000)	Textarea	Yes	Mô tả quyền lợi, giá trị tài trợ và đối tượng.
DieuKienXetHocBong	nvarchar(2000)	Textarea	Yes	Chi tiết các điều kiện bắt buộc để xét cấp.
TrangThai	nvarchar(50)	Chưa mở, Đang nhận hồ sơ, Tạm dừng, Đã đóng	No	Trạng thái vận hành của chương trình học bổng.

Bảng 3.6. Cấu trúc bảng HoSoHocBong (Hồ sơ ứng tuyển)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaHoSoHocBong	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã định danh hồ sơ ứng tuyển.
MaSinhVien	int	FOREIGN KEY -> SinhVien ON DELETE RESTRICT	No	Mã sinh viên nộp hồ sơ.
MaHocBong	int	FOREIGN KEY -> HocBong ON DELETE RESTRICT	No	Mã chương trình học bổng ứng tuyển.
NgayNop	datetime2	Hệ thống tự động xác định	No	Thời điểm nộp hồ sơ (không cho SV tự sửa).
ThuXinHocBong	nvarchar(2000)	StringLength(2000)	No	Thư bày tỏ nguyện vọng và lý do xin học bổng.
GhiChu	nvarchar(1000)	StringLength(1000)	Yes	Ghi chú bổ sung hoặc tài liệu nộp kèm.
TrangThai	nvarchar(50)	8 Trạng thái chuẩn State Machine	No	Trạng thái hiện tại của hồ sơ.
NgayXuLy	datetime2	DateTime	Yes	Thời điểm cán bộ thẩm định sơ bộ hồ sơ.
NhanXetXetDuyet	nvarchar(1000)	Textarea	Yes	Đánh giá, nhận xét sơ tuyển của cán bộ.
FileMinhChung	nvarchar(1000)	URL Cloud Storage	Yes	Đường dẫn file bảng điểm / chứng chỉ tải lên.

Bảng 3.7. Cấu trúc bảng LichPhongVan (Lịch phỏng vấn)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaLichPhongVan	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã định danh buổi phỏng vấn.
MaHoSoHocBong	int	FOREIGN KEY -> HoSoHocBong	No	Mã hồ sơ ứng tuyển được xếp lịch.
ThoiGianBatDau	datetime2	Required, datetime-local	No	Thời điểm bắt đầu buổi phỏng vấn.
ThoiGianKetThuc	datetime2	Required, > ThoiGianBatDau	No	Thời điểm kết thúc buổi phỏng vấn.
HinhThucPhongVan	nvarchar(100)	Trực tiếp, Trực tuyến	Yes	Hình thức tổ chức buổi phỏng vấn.
DiaDiemHoacLienKet	nvarchar(500)	StringLength(500)	Yes	Địa chỉ phòng họp thực tế hoặc đường link Meet.
NguoiPhongVan	nvarchar(100)	Required	No	Tên cán bộ hoặc Hội đồng phỏng vấn.
GhiChu	nvarchar(1000)	StringLength(1000)	Yes	Yêu cầu trang phục, tài liệu mang theo.
TrangThai	nvarchar(50)	Đã lên lịch, Đã hoàn thành, Đã hủy	No	Tình trạng tiến độ của buổi phỏng vấn.

Bảng 3.8. Cấu trúc bảng KetQuaHocBong (Kết quả xét học bổng)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaKetQuaHocBong	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã định danh kết quả xét học bổng.
MaHoSoHocBong	int	UNIQUE FOREIGN KEY -> HoSoHocBong	No	Mã hồ sơ liên kết 1-1 với kết quả.
KetQua	nvarchar(100)	Được cấp học bổng / Không được cấp	No	Quyết định phê duyệt cuối cùng.
DiemDanhGia	decimal(5,2)	Range(0, 100.0)	Yes	Điểm đánh giá buổi phỏng vấn (thang điểm 100).
NgayCapNhat	datetime2	Hệ thống tự động xác định	No	Thời điểm công bố quyết định trao học bổng.

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
NhanXet	nvarchar(1000)	Textarea	Yes	Nhận xét tổng kết và định hướng của Hội đồng.

Bảng 3.9. Cấu trúc bảng ThongBao (Thông báo Realtime)

Tên cột	Kiểu dữ liệu SQL	Khóa / Ràng buộc	Nullable	Mô tả nghiệp vụ
MaThongBao	int	PRIMARY KEY, IDENTITY(1,1)	No	Mã định danh bản ghi thông báo.
MaTaiKhoan	int	FOREIGN KEY -> TaiKhoan	No	Mã tài khoản người nhận thông báo.
MaSinhVien	int	FOREIGN KEY -> SinhVien	Yes	Mã sinh viên nhận tin (nếu có).
TieuDe	nvarchar(250)	Required	No	Tiêu đề thông báo hiển thị trên chuông.
NoiDung	nvarchar(2000)	Required	No	Nội dung chi tiết thông báo.
LoaiThongBao	nvarchar(50)	Hồ sơ, Phỏng vấn, Kết quả...	Yes	Phân loại để hiển thị icon phù hợp.
LienKet	nvarchar(500)	URL Link	Yes	Đường dẫn điều hướng khi nhấp vào thông báo.
MaHoSoHocBong	int	FOREIGN KEY -> HoSoHocBong	Yes	Mã hồ sơ liên quan trực tiếp.
DaDoc	bit	Default: 0 (Chưa đọc)	No	Đánh dấu trạng thái đã đọc của người dùng.
NgayTao	datetime2	DateTime.Now	No	Thời điểm sinh thông báo.

3.4. Dữ liệu mẫu (Database Seeding) và DbInitializer

Tuân thủ chặt chẽ Mục 16 của đề cương, lớp DbInitializer.cs tự động kiểm tra và khởi tạo sẵn bộ dữ liệu mẫu vô cùng phong phú ngay trong lần chạy đầu tiên:

Tài khoản kiểm thử: 01 Quản trị viên (admin), 02 Cán bộ học bổng (canbo, canbo1) và 30 tài khoản Sinh viên tương ứng từ sv001 đến sv030 với mật khẩu băm mặc định '123456'.

Đơn vị tài trợ: 05 Đơn vị tài trợ lớn: Quỹ Khuyến học Vingroup, Tập đoàn Viettel, FPT Software, Vietcombank, Samsung Electronics Việt Nam.

Học bổng: 15 Chương trình học bổng thuộc nhiều bậc đào tạo (Đại học, Cao đẳng) với đầy đủ 4 trạng thái: Đang nhận hồ sơ (8 học bổng), Chưa mở (2 học bổng), Tạm dừng (1 học bổng) và Đã đóng/Hết hạn (4 học bổng), đảm bảo phân trang sinh động.

Sinh viên: 30 Sinh viên thuộc các ngành CNTT, Hệ thống thông tin, Kỹ thuật phần mềm với điểm GPA trải rộng từ 2.45 đến 3.95.

Hồ sơ học bổng: 42 Hồ sơ ứng tuyển trải đều đầy đủ cả 8 trạng thái (Chờ duyệt, Đạt xét sơ bộ, Không đạt sơ bộ, Chờ phỏng vấn, Đã phỏng vấn, Được cấp học bổng, Không được cấp học bổng, Đã hủy).

Lịch phỏng vấn: 18 Buổi phỏng vấn cả hình thức Trực tiếp và Online Google Meet.

Kết quả học bổng: Dữ liệu kết quả phỏng vấn đầy đủ cả Được cấp và Không được cấp kèm điểm số và nhận xét chi tiết.

CHƯƠNG 4: TRIỂN KHAI CHI TIẾT 05 MODULE VÀ MÃ NGUỒN QUAN TRỌNG

[Quay lại Menu điều hướng]

4.1. Module 1: Tài khoản, Đăng nhập, Phân quyền và Đơn vị tài trợ (SV1 - Nguyễn Tuấn Trung)

Module 1 chịu trách nhiệm bảo mật vòng ngoài, quản lý danh tính người dùng và danh mục đối tác tài trợ của trường:

4.1.1. Cơ chế Đăng nhập, Đăng xuất và Quản lý phiên Session

Khi người dùng gửi biểu mẫu đăng nhập (POST: TaiKhoan/DangNhap), hệ thống sử dụng LINQ để truy vấn tài khoản theo TenDangNhap hoặc Email, sau đó kiểm tra mật khẩu qua lớp PasswordHelper và kiểm tra cờ trạng thái TrangThai (tài khoản không bị khóa):

Mã nguồn minh họa: Xác thực đăng nhập và nạp Session tại TaiKhoanController.cs

```
// Trích đoạn xử lý đăng nhập tại Controllers/TaiKhoanController.cs:
var user = await _context.TaiKhoan
    .Include(t => t.SinhVien)
    .FirstOrDefaultAsync(t => t.TenDangNhap == model.TenDangNhap || t.Email ==
model.TenDangNhap);

if (user == null || !PasswordHelper.VerifyPassword(model.MatKhau, user.MatKhau))
{
    ModelState.AddModelError(string.Empty, "Tài khoản hoặc mật khẩu không chính
xác.");
    return View(model);
}

if (!user.TrangThai)
{
    ModelState.AddModelError(string.Empty, "Tài khoản của bạn đã bị khóa. Vui
lòng liên hệ quản trị viên.");
    return View(model);
}

// Lưu trữ thông tin tối thiểu vào Session theo Mục 5.2 của đề tài:
HttpContext.Session.SetInt32(AppConstants.SessionKeys.MaTaiKhoan,
user.MaTaiKhoan);
HttpContext.Session.SetString(AppConstants.SessionKeys.TenDangNhap,
user.TenDangNhap);
HttpContext.Session.SetString(AppConstants.SessionKeys.HoTen, user.HoTen);
HttpContext.Session.SetString(AppConstants.SessionKeys.VaiTro, user.VaiTro);

if (user.SinhVien != null)
{
    HttpContext.Session.SetInt32(AppConstants.SessionKeys.MaSinhVien,
user.SinhVien.MaSinhVien);
}
```

```
// Cấp phát JWT Access Token (15 phút) & Refresh Token (7 ngày) vào 2 cookie
HttpOnly
if (_tokenAuthService != null)
{
    await _tokenAuthService.IssueTokensAsync(user, HttpContext);
}
```

4.1.2. Phân quyền trên Controller bằng RoleAuthorizeAttribute

Để đáp ứng yêu cầu nghiêm ngặt tại Mục 5.4 ('Các chức năng quản trị phải kiểm tra quyền tại Controller trước khi xử lý, không được chỉ ẩn menu hoặc nút trên View'), nhóm đã tự xây dựng lớp ActionFilter RoleAuthorizeAttribute kế thừa ActionFilterAttribute:

Mã nguồn minh họa: Bộ lọc phân quyền RoleAuthorizeAttribute bảo vệ Controller

```
// Trích đoạn bộ lọc Attributes/RoleAuthorizeAttribute.cs:
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method, AllowMultiple =
true, Inherited = true)]
public class RoleAuthorizeAttribute : ActionFilterAttribute
{
    private readonly string[] _allowedRoles;
    public RoleAuthorizeAttribute(params string[] allowedRoles) => _allowedRoles
= allowedRoles;

    public override void OnActionExecuting(ActionExecutingContext context)
    {
        var session = context.HttpContext.Session;
        var maTaiKhoan = session.GetInt32(AppConstants.SessionKeys.MaTaiKhoan);
        var vaiTro = session.GetString(AppConstants.SessionKeys.VaiTro);

        // 1. Kiểm tra đăng nhập
        if (!maTaiKhoan.HasValue || string.IsNullOrEmpty(vaiTro))
        {
            var returnUrl = context.HttpContext.Request.Path +
context.HttpContext.Request.QueryString;
            context.Result = new RedirectToActionResult("DangNhap", "TaiKhoan",
new { returnUrl });
            return;
        }

        // 2. Kiểm tra vai trò được phép
        if (_allowedRoles != null && _allowedRoles.Length > 0)
        {
            var hasPermission = _allowedRoles.Any(r => IsRoleMatch(r, vaiTro));
            if (!hasPermission)
            {
                context.Result = new RedirectToActionResult("AccessDenied",
"TaiKhoan", null);
                return;
            }
        }
        base.OnActionExecuting(context);
    }
}
```

4.1.3. Cơ chế Xoay vòng Refresh Token trong Transaction

Khi Access Token hết hạn, phương thức RefreshTokensInTransactionAsync trong TokenAuthService được kích hoạt để thực hiện xoay vòng token bên trong một Database Transaction:

Mã nguồn minh họa: Xoay vòng Refresh Token an toàn trong Transaction

```
// Trích đoạn xoay vòng token tại Services/Auth/TokenAuthService.cs:
await using var transaction = await
_context.Database.BeginTransactionAsync(cancellationToken);

var storedToken = await _context.RefreshToken
    .Include(r => r.User).ThenInclude(u => u!.SinhVien)
    .FirstOrDefaultAsync(r => r.Token == refreshToken.Trim(), cancellationToken);

if (storedToken == null || storedToken.ExpiresAt <= DateTime.UtcNow ||
!storedToken.User!.TrangThai)
{
    if (storedToken != null) {
        _context.RefreshToken.Remove(storedToken);
        await _context.SaveChangesAsync(cancellationToken);
    }
    await transaction.CommitAsync(cancellationToken);
    return null;
}

// Xoay vòng (Rotation): Xóa token cũ và thêm token mới vào CSDL trong cùng
Transaction
var user = storedToken.User;
_context.RefreshToken.Remove(storedToken);

var newRefreshTokenString = GeneratePlainRefreshTokenString();
_context.RefreshToken.Add(new RefreshToken {
    Token = newRefreshTokenString,
    UserId = user.MaTaiKhoan,
    ExpiresAt = DateTime.UtcNow.AddDays(_options.RefreshTokenDays),
    CreatedAt = DateTime.UtcNow
});

await _context.SaveChangesAsync(cancellationToken);
await transaction.CommitAsync(cancellationToken);

var newAccessToken = GenerateAccessToken(user, out var newAccessExpiresUtc);
SetAuthCookies(httpContext, newAccessToken, newAccessExpiresUtc,
newRefreshTokenString, DateTime.UtcNow.AddDays(7));
```

4.1.4. Quản lý Đơn vị tài trợ và Kiểm tra ràng buộc xóa

Tại DonViTaiTroController.cs, quản trị viên có thể xem danh sách (kèm tìm kiếm và phân trang), thêm mới, chỉnh sửa thông tin. Đặc biệt, khi xóa một đơn vị tài trợ, hệ thống dùng LINQ kiểm tra ràng buộc xem đã có học bổng liên kết hay chưa:

Mã nguồn minh họa: Kiểm tra ràng buộc xóa đơn vị tài trợ bằng LINQ

```
// Kiểm tra ràng buộc khi xóa Đơn vị tài trợ tại DonViTaiTroController.cs:
bool hasLinkedScholarships = await _context.HocBong.AnyAsync(h => h.MaDonViTaiTro
```

```
== id);
if (hasLinkedScholarships)
{
    TempData["ErrorMessage"] = "Không thể xóa đơn vị tài trợ này vì đang có các
chương trình học bổng liên quan.";
    return RedirectToAction(nameof(Index));
}
_context.DonViTaiTro.Remove(donViTaiTro);
await _context.SaveChangesAsync();
```

4.2. Module 2: Quản lý học bổng, Tìm kiếm, Lọc, Sắp xếp và Phân trang LINQ (SV2 - Nguyễn Văn Tình)

Module 2 thực hiện nghiệp vụ cốt lõi: quản lý toàn bộ các chương trình học bổng và cung cấp bộ công cụ tra cứu mạnh mẽ kết hợp 4 thành phần Tìm kiếm - Lọc - Sắp xếp - Phân trang trên cùng một luồng truy vấn duy nhất:

4.2.1. Chuỗi truy vấn kết hợp Search + Filter + Sort + Pagination

Tại HocBongController.cs (Action Index), truy vấn được xây dựng theo mô hình Deferred Execution trên giao diện IQueryable<HocBong>, chỉ thực thi xuống CSDL khi phân trang Skip/Take:

Mã nguồn minh họa: Truy vấn kết hợp Search + Filter + Sort + Pagination tại HocBongController.cs

```
// Trích đoạn chuỗi truy vấn kết hợp tại Controllers/HocBongController.cs:
var query = _context.HocBong
    .Include(h => h.DonViTaiTro)
    .Include(h => h.HoSoHocBongs)
    .AsQueryable();

// 1. TÌM KIẾM THEO TÊN HỌC BỔNG HOẶC ĐƠN VỊ TÀI TRỢ
if (!string.IsNullOrWhiteSpace(searchString))
{
    var keyword = searchString.Trim();
    query = query.Where(h => h.TenHocBong.Contains(keyword) ||
                             (h.DonViTaiTro != null &&
h.DonViTaiTro.TenDonViTaiTro.Contains(keyword)));
}

// 2. LỌC ĐA TIÊU CHÍ KẾT HỢP
if (donViTaiTroId.HasValue && donViTaiTroId.Value > 0)
    query = query.Where(h => h.MaDonViTaiTro == donViTaiTroId.Value);

if (!string.IsNullOrWhiteSpace(bacDaoTao))
    query = query.Where(h => h.BacDaoTao == bacDaoTao);

if (!string.IsNullOrWhiteSpace(trangThai))
    query = query.Where(h => h.TrangThai == trangThai);

if (minGpa.HasValue)
    query = query.Where(h => h.DiemTrungBinhToiThieu <= minGpa.Value);
```

```
var today = DateTime.Today;
if (tinhTrangHan == "ConHan")
    query = query.Where(h => h.HanNopHoSo >= today && h.NgayBatDauNhanHoSo <=
today && h.TrangThai == "Đang nhận hồ sơ");
else if (tinhTrangHan == "HetHan")
    query = query.Where(h => h.HanNopHoSo < today || h.TrangThai == "Đã đóng");

// 3. SẮP XẾP ĐỘNG 6 CHẾ ĐỘ BẰNG SWITCH EXPRESSION
query = sortOrder switch
{
    "name_asc"   => query.OrderBy(h => h.TenHocBong),
    "name_desc"  => query.OrderByDescending(h => h.TenHocBong),
    "date_asc"   => query.OrderBy(h => h.HanNopHoSo),
    "date_desc"  => query.OrderByDescending(h => h.HanNopHoSo),
    "slots_asc"  => query.OrderBy(h => h.SoSuat),
    "slots_desc" => query.OrderByDescending(h => h.SoSuat),
    _            => query.OrderByDescending(h => h.MaHocBong)
};

// 4. PHÂN TRANG SERVER-SIDE VỚI SKIP / TAKE (KẾT HỢP HYBRID REDIS CACHE)
int pageSize = 6;
int totalItems = await query.CountAsync();
var items = await query.AsNoTracking()
                       .Skip((page - 1) * pageSize)
                       .Take(pageSize)
                       .ToListAsync();
```

4.2.2. Kỹ thuật duy trì bộ lọc và tham số khi chuyển trang

Để đáp ứng yêu cầu 'Khi chuyển trang phải giữ lại từ khóa, bộ lọc và kiểu sắp xếp đang sử dụng', toàn bộ giá trị tìm kiếm, mã đơn vị tài trợ, bậc đào tạo, tình trạng hạn nộp, trạng thái sắp xếp được nạp vào ViewBag và truyền trực tiếp vào các thẻ liên kết phân trang trên View:

Mã nguồn minh họa: Truyền tham số duy trì bộ lọc trên liên kết phân trang Razor View

```
<!-- Trích đoạn phân trang duy trì tham số tại Views/HocBong/Index.cshtml: -->
<a class="page-link" asp-action="Index"
   asp-route-page="@(ViewBag.CurrentPage + 1)"
   asp-route-searchString="@ViewBag.CurrentSearch"
   asp-route-donViTaiTroId="@ViewBag.CurrentDonVi"
   asp-route-bacDaoTao="@ViewBag.CurrentBac"
   asp-route-trangThai="@ViewBag.CurrentTrangThai"
   asp-route-minGpa="@ViewBag.CurrentMinGpa"
   asp-route-tinhTrangHan="@ViewBag.CurrentTinhTrangHan"
   asp-route-sortOrder="@ViewBag.CurrentSort">
   Trang sau &raquo;
</a>
```

4.2.3. Kiểm soát nghiệp vụ Xóa học bổng

Khi học bổng đã có hồ sơ xin học bổng của sinh viên, hệ thống ngăn chặn việc xóa vật lý để không làm mất dữ liệu lịch sử:

Mã nguồn minh họa: Chặn xóa học bổng khi đã có hồ sơ đăng ký

```
// Kiểm tra ràng buộc khi xóa học bổng tại HocBongController.cs:
bool hasApplications = await _context.HoSoHocBong.AnyAsync(hs => hs.MaHocBong ==
id);
if (hasApplications)
{
    TempData["ErrorMessage"] = "Không thể xóa học bổng này vì đã có hồ sơ của
sinh viên nộp. Vui lòng chuyển trạng thái sang 'Đã đóng' để bảo toàn dữ liệu lịch
sử.";
    return RedirectToAction(nameof(Index));
}
_context.HocBong.Remove(hocBong);
await _context.SaveChangesAsync();
```

4.3. Module 3: Quản lý sinh viên, Hồ sơ cá nhân và Nộp hồ sơ xin học bổng (SV3 - Nguyễn Phi Hùng)

Module 3 phục vụ đối tượng người dùng Sinh viên, bao gồm hồ sơ học vấn cá nhân và nghiệp vụ nộp/hủy/theo dõi hồ sơ ứng tuyển học bổng:

4.3.1. Kiểm tra 03 điều kiện tiên quyết khi Nộp hồ sơ bằng LINQ

Tại HoSoHocBongController.cs (Action NopHoSo), trước khi hiển thị form và trước khi lưu bản ghi vào CSDL, hệ thống kiểm tra 03 quy tắc nghiệp vụ bắt buộc:

Mã nguồn minh họa: Kiểm tra 03 điều kiện nghiệp vụ nộp hồ sơ bằng LINQ

```
// Trích đoạn kiểm tra điều kiện nộp hồ sơ tại HoSoHocBongController.cs:
var today = DateTime.Today;

// 1. Kiểm tra học bổng đang nhận hồ sơ & còn hạn nộp
if (hocBong.TrangThai != "Đang nhận hồ sơ" || hocBong.NgayBatDauNhanHoSo > today
|| hocBong.HanNopHoSo < today)
{
    TempData["ErrorMessage"] = "Học bổng này hiện không trong thời gian tiếp nhận
hồ sơ.";
    return RedirectToAction("Index", "HocBong");
}

// 2. Kiểm tra không được nộp trùng 2 hồ sơ đang xử lý cho cùng 1 học bổng
bool hasActiveApp = await _context.HoSoHocBong.AnyAsync(hs =>
    hs.MaSinhVien == sinhVien.MaSinhVien &&
    hs.MaHocBong == maHocBong &&
    hs.TrangThai != "Đã hủy");
if (hasActiveApp)
{
    TempData["ErrorMessage"] = "Bạn đã có một hồ sơ đang được xử lý cho chương
trình học bổng này.";
    return RedirectToAction("HoSoCuaToi");
}

// 3. Kiểm tra điều kiện điểm GPA tối thiểu của sinh viên
if (sinhVien.DiemTrungBinhTichLuy < hocBong.DiemTrungBinhToiThieu)
{
```

```
    TempData["ErrorMessage"] = $"Điểm GPA tích lũy của bạn
({sinhVien.DiemTrungBinhTichLuy}) chưa đạt mức tối thiểu yêu cầu
({hocBong.DiemTrungBinhToiThieu}) của học bổng này.";
    return RedirectToAction("Details", "HocBong", new { id = maHocBong });
}

// Khởi tạo hồ sơ: NgayNop lấy từ hệ thống, trạng thái mặc định Chờ duyệt
var hoSo = new HoSoHocBong {
    MaSinhVien = sinhVien.MaSinhVien,
    MaHocBong = hocBong.MaHocBong,
    NgayNop = DateTime.Now,
    ThuXinHocBong = ThuXinHocBong,
    GhiChu = GhiChu ?? "",
    TrangThai = "Chờ duyệt",
    FileMinhChung = uploadedFileName
};
_context.HoSoHocBong.Add(hoSo);
await _context.SaveChangesAsync();
```

4.3.2. Nghiệp vụ Hủy hồ sơ ứng tuyển hợp lệ

Sinh viên chỉ được quyền hủy hồ sơ của chính mình khi hồ sơ đang ở trạng thái Chờ duyệt hoặc Đạt xét duyệt sơ bộ nhưng CHƯA có lịch phỏng vấn:

Mã nguồn minh họa: Xử lý hủy hồ sơ ứng tuyển theo điều kiện FSM

```
// Kiểm tra điều kiện hủy hồ sơ tại HoSoHocBongController.cs:
bool hasScheduledInterview = hoSo.LichPhongVans.Any(l => l.TrangThai != "Đã
hủy");
if (hoSo.TrangThai != "Chờ duyệt" && (hoSo.TrangThai != "Đạt xét duyệt sơ bộ" ||
hasScheduledInterview))
{
    TempData["ErrorMessage"] = "Không thể hủy hồ sơ này do hồ sơ đã có lịch phỏng
vấn hoặc đã có kết quả xét duyệt cuối cùng.";
    return RedirectToAction(nameof(HoSoCuaToi));
}

hoSo.TrangThai = "Đã hủy";
await _context.SaveChangesAsync();
```

4.3.3. Bảo mật quyền dữ liệu cá nhân của Sinh viên

Tại action Details của HoSoHocBongController, hệ thống lấy MaTaiKhoan từ Session để kiểm tra quyền sở hữu; nếu sinh viên cố ý sửa tham số ID trên URL để xem hồ sơ của bạn học khác, Controller lập tức trả về mã lỗi HTTP 403 Forbid():

Mã nguồn minh họa: Bảo mật quyền dữ liệu cá nhân sinh viên

```
// Bảo mật dữ liệu: Chặn sinh viên xem trộm hồ sơ người khác
if ("Sinh viên".Equals(userRole, StringComparison.OrdinalIgnoreCase))
{
    var maTaiKhoan = HttpContext.Session.GetInt32("MaTaiKhoan")!.Value;
    var currentSv = await _context.SinhVien.FirstOrDefaultAsync(s => s.MaTaiKhoan
== maTaiKhoan);
    if (currentSv == null || hoSo.MaSinhVien != currentSv.MaSinhVien)
    {
        return Forbid(); // Từ chối truy cập HTTP 403
```

```
    }
}
```

4.4. Module 4: Tiếp nhận hồ sơ, Xét duyệt sơ bộ và Quản lý luồng trạng thái (SV4 - Phan Sỹ Tú)

Module 4 cung cấp không gian làm việc chuyên nghiệp cho Cán bộ học bổng để thẩm định các hồ sơ ứng tuyển:

4.4.1. Danh sách hồ sơ cần xử lý

Tại HoSoHocBongController.cs (Index), cán bộ có thể tìm kiếm hồ sơ theo họ tên sinh viên hoặc tên học bổng, lọc theo chương trình học bổng, đơn vị tài trợ hoặc trạng thái, và sắp xếp theo ngày nộp tăng/giảm hoặc tên sinh viên.

4.4.2. Xét duyệt sơ bộ và Tự động hóa thông báo

Cán bộ chỉ được phép xét duyệt các hồ sơ có trạng thái 'Chờ duyệt'. Khi đánh giá Đạt hoặc Không đạt, hệ thống tự động ghi nhận NgayXuLy và NhanXetXetDuyet, đồng thời gửi email thông báo và đẩy thông báo Realtime xuống chuông thông báo của sinh viên:

Mã nguồn minh họa: Xét duyệt sơ bộ và chuyển tiếp sang lập lịch phỏng vấn

```
// Trích đoạn xét duyệt sơ bộ tại HoSoHocBongController.cs (POST: XetDuyet):
if (hoSo.TrangThai != "Chờ duyệt")
{
    TempData["ErrorMessage"] = "Hồ sơ không ở trạng thái Chờ duyệt.";
    return RedirectToAction(nameof(Index));
}

hoSo.TrangThai = KetQuaSoBo; // "Đạt xét duyệt sơ bộ" hoặc "Không đạt xét duyệt
sơ bộ"
hoSo.NgayXuLy = DateTime.Now; // Hệ thống tự động ghi nhận
hoSo.NhanXetXetDuyet = NhanXetXetDuyet ?? "Đạt tiêu chuẩn sơ tuyển";
await _context.SaveChangesAsync();

// Tự động gửi email thông báo kết quả sơ bộ qua hàng đợi Brevo API
if (!string.IsNullOrWhiteSpace(hoSo.SinhVien?.Email))
{
    await _emailService.SendEmailAsync(hoSo.SinhVien.Email, subject, message);
}

// Gửi thông báo thời gian thực SignalR
await _notificationService.GuiThongBaoAsync(
    hoSo.SinhVien.MaTaiKhoan, hoSo.MaSinhVien, notifTitle, notifDesc, "Hồ sơ",
    $"/HoSoHocBong/Details/{hoSo.MaHoSoHocBong}", hoSo.MaHoSoHocBong
);

// Nếu đạt sơ tuyển, tự động chuyển hướng cán bộ sang màn hình lập lịch phỏng vấn
if (KetQuaSoBo == "Đạt xét duyệt sơ bộ")
{
```

```
    return RedirectToAction("Create", "LichPhongVan", new { maHoSo =
hoSo.MaHoSoHocBong });
}
```

4.5. Module 5: Lịch phỏng vấn, Kết quả, Dashboard và Thống kê (SV5 - Vũ Mạnh Quỳnh)

Module 5 giải quyết các bài toán thuật toán và tổng hợp dữ liệu phức tạp nhất trong toàn bộ hệ thống:

4.5.1. Thuật toán LINQ kiểm tra chống trùng lịch phỏng vấn

Một trong những đóng góp thuật toán nổi bật của đề tài nằm tại LichPhongVanController.cs. Hai khoảng thời gian [StartA, EndA] và [StartB, EndB] giao nhau (bị trùng lịch) khi và chỉ khi: StartA < EndB và EndA > StartB. Hệ thống áp dụng toán tử AnyAsync của LINQ để kiểm tra chống trùng độc lập cho cả Sinh viên lẫn Hội đồng phỏng vấn:

Mã nguồn minh họa: Thuật toán LINQ kiểm tra chống trùng lịch phỏng vấn

```
// Trích đoạn thuật toán LINQ chống trùng lịch tại LichPhongVanController.cs:
// Kiểm tra 1: Thời gian kết thúc phải sau thời gian bắt đầu
if (lich.ThoiGianKetThuc <= lich.ThoiGianBatDau)
{
    ModelState.AddModelError("ThoiGianKetThuc", "Thời gian kết thúc phỏng vấn
phải sau thời gian bắt đầu.");
}

// Kiểm tra 2: THUẬT TOÁN LINQ KIỂM TRA TRÙNG LỊCH CHO SINH VIÊN
bool isStudentConflict = await _context.LichPhongVan
    .Include(l => l.HoSoHocBong)
    .AnyAsync(l =>
        l.HoSoHocBong.MaSinhVien == hoSo.MaSinhVien &&
        l.TrangThai != "Đã hủy" &&
        l.ThoiGianBatDau < lich.ThoiGianKetThuc &&
        l.ThoiGianKetThuc > lich.ThoiGianBatDau);

if (isStudentConflict)
{
    ModelState.AddModelError("ThoiGianBatDau", $"Sinh viên
'{hoSo.SinhVien?.HoTen}' đã có một lịch phỏng vấn khác bị trùng khoảng thời gian
này.");
}

// Kiểm tra 3: THUẬT TOÁN LINQ KIỂM TRA TRÙNG LỊCH CHO HỘI ĐỒNG PHỎNG VẤN
if (!string.IsNullOrWhiteSpace(lich.NguoiPhongVan))
{
    bool isInterviewerConflict = await _context.LichPhongVan.AnyAsync(l =>
        l.NguoiPhongVan == lich.NguoiPhongVan &&
        l.TrangThai != "Đã hủy" &&
        l.ThoiGianBatDau < lich.ThoiGianKetThuc &&
        l.ThoiGianKetThuc > lich.ThoiGianBatDau);

    if (isInterviewerConflict)
```

```
    {
        ModelState.AddModelError("NguoiPhongVan", $"Người/Hội đồng phỏng vấn
'{lich.NguoiPhongVan}' đã có một lịch khác bị trùng khoảng thời gian này.");
    }
}

// Khi lập lịch thành công: Hồ sơ tự động chuyển sang 'Chờ phỏng vấn'
hoSo.TrangThai = "Chờ phỏng vấn";
_context.Add(lich);
await _context.SaveChangesAsync();
```

4.5.2. Công bố kết quả xét học bổng và Kiểm soát số suất

Tại KetQuaHocBongController.cs, cán bộ nhập điểm đánh giá phỏng vấn (thang điểm 0 - 100) và lựa chọn kết quả. Hệ thống sử dụng LINQ để đếm tổng số hồ sơ đã được cấp học bổng nhằm cảnh báo khi đạt đủ số suất:

Mã nguồn minh họa: Kiểm soát số suất học bổng bằng LINQ CountAsync

```
// Kiểm soát số suất học bổng bằng LINQ tại KetQuaHocBongController.cs:
int soDaCap = await _context.HoSoHocBong.CountAsync(h => h.MaHocBong ==
hoSo.MaHocBong && h.TrangThai == "Được cấp học bổng");
ViewBag.SoDaCap = soDaCap;
ViewBag.TongSoSuat = hoSo.HocBong.SoSuat;
ViewBag.CanhBaoHetSuat = soDaCap >= hoSo.HocBong.SoSuat;

// Cập nhật kết quả và đồng bộ trạng thái hồ sơ theo State Machine:
hoSo.TrangThai = KetQua; // "Được cấp học bổng" hoặc "Không được cấp học bổng"
await _context.SaveChangesAsync();
```

4.5.3. Dashboard tổng hợp 9 chỉ số quản trị và Dashboard sinh viên

Tại HomeController.cs, hệ thống tự động phân loại vai trò người dùng: Nếu là Sinh viên, hiển thị giao diện StudentDashboard với thống kê số hồ sơ đã nộp, hồ sơ đang xét duyệt, học bổng đã nhận, lịch phỏng vấn sắp tới. Nếu là Cán bộ/Admin, hiển thị Dashboard quản trị với 9 chỉ số bắt buộc theo mục 9.5 của đề cương:

Mã nguồn minh họa: Truy vấn 9 chỉ số Dashboard quản trị bằng LINQ

```
// Thống kê 9 chỉ số Dashboard quản trị bằng LINQ tại HomeController.cs:
var tongDonViTaiTro = await _context.DonViTaiTro.CountAsync();
var tongHocBong = await _context.HocBong.CountAsync();
var hocBongDangNhan = await _context.HocBong.CountAsync(h => h.TrangThai == "Đang
nhận hồ sơ");
var tongSinhVien = await _context.SinhVien.CountAsync();
var tongHoSo = await _context.HoSoHocBong.CountAsync();
var hoSoChoDuyet = await _context.HoSoHocBong.CountAsync(hs => hs.TrangThai ==
"Chờ duyệt");
var hoSoDatSoBo = await _context.HoSoHocBong.CountAsync(hs => hs.TrangThai ==
"Đạt xét duyệt sơ bộ");
var lichPVSapToi = await _context.LichPhongVan.CountAsync(l => l.TrangThai == "Đã
lên lịch");
var hoSoDuocCap = await _context.HoSoHocBong.CountAsync(hs => hs.TrangThai ==
"Được cấp học bổng");
```

4.5.4. Báo cáo thống kê 10 chỉ tiêu nghiệp vụ bằng LINQ

Tại ThongKeController.cs, nhóm đã triển khai đầy đủ 10/10 chỉ tiêu thống kê theo Mục 9.6 của đề cương bằng các toán tử LINQ nâng cao (GroupBy, OrderByDescending, FirstOrDefault, Select):

Mã nguồn minh họa: 10 Chỉ tiêu thống kê bằng LINQ theo Mục 9.6 của đề cương

```
// 10 Chỉ tiêu thống kê LINQ tại Controllers/ThongKeController.cs:
// 1. Số học bổng theo đơn vị tài trợ
vm.HocBongTheoDonVi = await _context.HocBong
    .GroupBy(h => h.DonViTaiTro!.TenDonViTaiTro)
    .Select(g => new ThongKeItem { Ten = g.Key, SoLuong = g.Count() })
    .OrderByDescending(x => x.SoLuong).ToListAsync();

// 2. Số hồ sơ theo từng học bổng
vm.HoSoTheoHocBong = await _context.HoSoHocBong
    .GroupBy(hs => hs.HocBong!.TenHocBong)
    .Select(g => new ThongKeItem { Ten = g.Key, SoLuong = g.Count() })
    .OrderByDescending(x => x.SoLuong).Take(8).ToListAsync();

// 3. Số hồ sơ theo trạng thái
vm.HoSoTheoTrangThai = await _context.HoSoHocBong
    .GroupBy(hs => hs.TrangThai)
    .Select(g => new ThongKeItem { Ten = g.Key, SoLuong = g.Count()
}).ToListAsync();

// 4. Số hồ sơ đạt xét duyệt sơ bộ theo học bổng
vm.HoSoDatSoBoTheoHocBong = await _context.HoSoHocBong
    .Where(hs => hs.TrangThai != "Chờ duyệt" && hs.TrangThai != "Không đạt xét
duyệt sơ bộ" && hs.TrangThai != "Đã hủy")
    .GroupBy(hs => hs.HocBong!.TenHocBong)
    .Select(g => new ThongKeItem { Ten = g.Key, SoLuong = g.Count()
}).ToListAsync();

// 5. Học bổng nhận nhiều hồ sơ nhất
var mostPopular = await _context.HoSoHocBong
    .GroupBy(hs => hs.HocBong!.TenHocBong)
    .Select(g => new { Ten = g.Key, Count = g.Count() })
    .OrderByDescending(x => x.Count).FirstOrDefaultAsync();

// 6. Số hồ sơ được cấp theo đơn vị tài trợ
vm.HoSoCapTheoDonVi = await _context.HoSoHocBong
    .Where(hs => hs.TrangThai == "Được cấp học bổng")
    .GroupBy(hs => hs.HocBong!.DonViTaiTro!.TenDonViTaiTro)
    .Select(g => new ThongKeItem { Ten = g.Key, SoLuong = g.Count()
}).ToListAsync();

// 7. Số hồ sơ nộp theo tháng
vm.HoSoTheoThang = allApps.GroupBy(d => new { d.Year, d.Month })
    .OrderBy(g => g.Key.Year).ThenBy(g => g.Key.Month)
    .Select(g => new ThongKeItem { Ten = $"T{g.Key.Month:D2}/{g.Key.Year}",
SoLuong = g.Count() }).ToList();

// 8. Số lịch phỏng vấn theo tháng
vm.LichPVTheoThang = allSchedules.GroupBy(d => new { d.Year, d.Month })
    .OrderBy(g => g.Key.Year).ThenBy(g => g.Key.Month)
    .Select(g => new ThongKeItem { Ten = $"T{g.Key.Month:D2}/{g.Key.Year}",
SoLuong = g.Count() }).ToList();

// 9. Tỷ lệ hồ sơ qua xét duyệt sơ bộ theo từng học bổng
```

```
vm.TyLeDatSoBoTheoHocBong = allHocBongs.Where(h => h.HoSoHocBongs.Any())
    .Select(h => new TyLeItem {
        TenHocBong = h.TenHocBong,
        TongHoSo = h.HoSoHocBongs.Count,
        SoDatSoBo = h.HoSoHocBongs.Count(hs => hs.TrangThai != "Chờ duyệt" &&
hs.TrangThai != "Không đạt xét duyệt sơ bộ"),
        TyLePhanTram = Math.Round((double)h.HoSoHocBongs.Count(hs => hs.TrangThai
!= "Chờ duyệt" && hs.TrangThai != "Không đạt xét duyệt sơ bộ") /
h.HoSoHocBongs.Count * 100, 1)
    }).ToList();

// 10. Tỷ lệ hồ sơ được cấp trên số hồ sơ đã phỏng vấn
vm.TyLeCapTrenPhongVan = vm.TongDaPhongVan > 0 ?
Math.Round((double)vm.TongDuocCap / vm.TongDaPhongVan * 100, 1) : 0;
```

4.6. Triển khai các thành phần hạ tầng Backend nâng cao

Dự án sở hữu giải pháp tự động xóa cache cũ (Cache Invalidation) vô cùng độc đáo được triển khai trực tiếp tại QuanLyHocBong_UNETI2_TI17A3HNContext.cs bằng cách ghi đè phương thức SaveChangesAsync():

Mã nguồn minh họa: Tự động Invalidate Cache trong DbContext.SaveChangesAsync()

```
// Tự động xóa phân vùng Cache khi CSDL thay đổi tại
Data/QuanLyHocBong_UNETI2_TI17A3HNContext.cs:
public override async Task<int> SaveChangesAsync(CancellationToken
cancellationToken = default)
{
    bool hasDomainChanges = ChangeTracker.Entries().Any(e =>
        (e.State == EntityState.Added || e.State == EntityState.Modified ||
e.State == EntityState.Deleted) &&
        e.Entity is not Models.RefreshToken);

    int affected = await base.SaveChangesAsync(cancellationToken);

    if (hasDomainChanges && affected > 0 && _cache != null)
    {
        // Tự động xóa sạch các phân vùng cache bị ảnh hưởng
        await _cache.InvalidateByPrefixAsync(CacheKeys.DonViTaiTroPrefix,
cancellationToken);
        await _cache.InvalidateByPrefixAsync(CacheKeys.HocBongPrefix,
cancellationToken);
        await _cache.InvalidateByPrefixAsync(CacheKeys.DashboardPrefix,
cancellationToken);
        await _cache.InvalidateByPrefixAsync(CacheKeys.ThongKePrefix,
cancellationToken);
    }
    return affected;
}
```

CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM, KIỂM THỬ VÀ HƯỚNG DẪN VẬN HÀNH

[Quay lại Menu điều hướng]

5.1. Yêu cầu môi trường và hướng dẫn cài đặt từng bước

Yêu cầu môi trường: Hệ thống yêu cầu môi trường phát triển và máy chủ vận hành đáp ứng các tiêu chuẩn kỹ thuật sau:

Hệ điều hành: Hệ điều hành Windows 10/11 64-bit hoặc Linux (Ubuntu 22.04 LTS / Debian 12).

SDK & Khung làm việc: Microsoft .NET 10 SDK (phiên bản 10.0.100 trở lên) và Visual Studio 2026 / Visual Studio Code.

Cơ sở dữ liệu: Microsoft SQL Server 2019/2022 hoặc phiên bản SQL Server Express (mặc định instance .\SQLEXPRESS).

Bộ nhớ đệm: Redis Server 7.x (tùy chọn; nếu không có kết nối Redis, hệ thống tự động fallback sang bộ nhớ MemoryCache trong RAM mà không phát sinh lỗi).

Các bước triển khai: Quy trình cài đặt và khởi chạy chương trình từ mã nguồn:

Bước 1 - Lấy mã nguồn: Mở terminal dòng lệnh, điều hướng vào thư mục dự án và chuyển sang nhánh Module_5: 'git checkout Module_5'.

Bước 2 - Cấu hình chuỗi kết nối: Mở tập tin appsettings.json, kiểm tra chuỗi kết nối tại 'ConnectionStrings:QuanLyHocBong_UNETI2_TI17A3HNContext' (mặc định trỏ tới Server=.\SQLEXPRESS;Database=QuanLyHocBong;Trusted_Connection=True;TrustServerCertificate=True).

Bước 3 - Biên dịch dự án: Thực thi lệnh 'dotnet restore' và 'dotnet build -c Release' để khôi phục toàn bộ các gói thư viện NuGet và biên dịch mã nguồn.

Bước 4 - Khởi chạy & Tự động Seed CSDL: Thực thi lệnh 'dotnet run'. Trong lần khởi chạy đầu tiên, lớp DbInitializer.Seed() sẽ tự động áp dụng

Migration, tạo lập 10 bảng trong CSDL QuanLyHocBong và nạp đầy đủ dữ liệu mẫu.

Bước 5 - Truy cập ứng dụng: Mở trình duyệt web và truy cập địa chỉ 'https://localhost:5001' hoặc 'http://localhost:5000' để trải nghiệm hệ thống.

5.2. Danh sách tài khoản kiểm thử theo từng vai trò

Bảng 5.1 cung cấp danh sách các tài khoản kiểm thử đã được nạp sẵn vào cơ sở dữ liệu để phục vụ công tác chấm thi và bảo vệ đồ án:

Vai trò	Tên đăng nhập	Mật khẩu	Họ tên người dùng	Chức năng và kịch bản kiểm thử trọng tâm
Admin	admin	123456	Quản trị viên Hệ thống	Toàn quyền quản trị tài khoản (tạo mới, khóa/mở khóa), đơn vị tài trợ, chương trình học bổng, theo dõi toàn bộ hồ sơ, xem Dashboard & Thống kê.
Cán bộ học bổng 1	canbo	123456	Nguyễn Thị B	Quản lý chương trình học bổng; tiếp nhận và xét duyệt sơ bộ hồ sơ; lập lịch phỏng vấn (kiểm tra chống trùng); cập nhật kết quả xét học bổng; xuất báo cáo CSV.
Cán bộ học bổng 2	canbo1	123456	Lê Hoàng Long	Kiểm thử phân quyền cán bộ xét duyệt thứ hai; kiểm tra chống trùng lịch giữa các cán bộ/hội đồng phỏng vấn khác nhau.
Sinh viên 1	sv001	123456	Trần Sinh Viên (GPA: 3.80)	Sinh viên tiêu biểu ngành Hệ thống thông tin; kiểm thử nộp hồ sơ, theo dõi hồ sơ đạt sơ bộ, xem lịch phỏng vấn và xem kết quả được cấp học bổng.
Sinh viên 2	sv002	123456	Lê Thị Mai Anh (GPA: 3.65)	Kiểm thử quy trình nộp hồ sơ mới trực tuyến; upload tệp minh chứng và kiểm thử chức năng hủy hồ sơ hợp lệ khi đang ở trạng thái Chờ duyệt.
Sinh viên 5	sv005	123456	Hoàng Văn Nam (GPA: 2.45)	Kiểm thử trường hợp bị hệ thống chặn nộp hồ sơ do điểm GPA (2.45) thấp hơn điểm tối thiểu yêu cầu (GPA >= 3.0) của chương trình học bổng.

Vai trò	Tên đăng nhập	Mật khẩu	Họ tên người dùng	Chức năng và kịch bản kiểm thử trọng tâm
Sinh viên 6	sv006	123456	Vũ Mạnh Quỳnh (GPA: 3.75)	Kiểm thử luồng sinh viên xem thông tin cá nhân, cập nhật ảnh thẻ đại diện lên Supabase Storage và nhận thông báo thời gian thực.

5.3. Kết quả thực thi bộ kiểm thử tự động xUnit (10/10 Test Cases Passed)

Nhóm đã xây dựng một dự án kiểm thử tự động độc lập tại thư mục tests/QuanLyHocBong.Tests sử dụng khung kiểm thử xUnit và Microsoft.NET.Test.Sdk. Khi thực thi lệnh 'dotnet test tests/QuanLyHocBong.Tests/QuanLyHocBong.Tests.csproj -c Release', toàn bộ 10/10 kịch bản kiểm thử chuyên sâu đều vượt qua thành công (100% Passed) trong thời gian 2.0 giây:

Mã nguồn minh họa: Kết quả thực thi kiểm thử tự động xUnit trên nhánh Module_5

```
Passed!  - Failed: 0, Passed: 10, Skipped: 0, Total: 10, Duration: 2 s - QuanLyHocBong.Tests.dll (net10.0)
```

Bảng 5.2 mô tả chi tiết 10 kịch bản kiểm thử tự động và kết quả đạt được:

STT	Tên phương thức kiểm thử (Test Case)	Nội dung xác minh kỹ thuật	Kết quả
1	CacheService_Supports_Hit_Miss_Expiration_And_PrefixInvalidation	Kiểm tra cơ chế đọc/ghi Cache-Aside, tự động hết hạn theo TTL và xóa sạch cache theo tiền tố (InvalidateByPrefix).	PASS
2	CacheAndRateLimit_Handle_RedisConnectionFailure_Gracefully	Kiểm tra cơ chế tự phục hồi: Khi ngắt kết nối Redis, hệ thống tự động fallback sang MemoryCache và không gây văng ứng dụng.	PASS
3	DistributedRateLimitingMiddleware_EnforcesLimits_And_ReturnsHttp429	Kiểm tra chặn yêu cầu vượt ngưỡng tần suất truy cập và trả về mã lỗi HTTP 429 Too Many Requests kèm Retry-After header.	PASS
4	MultiInstanceRateLimiting_SharesCounters_AcrossInstances	Kiểm tra chia sẻ đồng bộ bộ đếm giới hạn tần suất giữa nhiều máy chủ/instance khác nhau thông qua Redis Atomic Script.	PASS

STT	Tên phương thức kiểm thử (Test Case)	Nội dung xác minh kỹ thuật	Kết quả
5	EmailQueue_ValidatesPayload_And_BlocksSecretLeaks	Kiểm tra xác thực địa chỉ email người nhận hợp lệ và chủ động ngăn chặn rò rỉ khóa bảo mật/mật khẩu trong nội dung email.	PASS
6	EmailWorker_RetriesTransientFailure_And_MovesPermanentFailureToDlq	Kiểm tra cơ chế thử lại lũy thừa (Exponential Backoff tối đa 3 lần) khi Brevo API gián đoạn và chuyển Job hỏng vào Dead-Letter Queue.	PASS
7	EmailQueue_DeduplicatesIdenticalJobs_And_WorkerShutsDownGracefully	Kiểm tra chống gửi email trùng lặp trong cửa sổ thời gian (Deduplication) và dừng tiến trình ngầm an toàn (Graceful Shutdown).	PASS
8	TokenAuthService_IssuesJwtAndPlainRefreshToken_RotatesInTransaction_And_RevokesOnLogout	Kiểm tra cấp phát JWT 15 phút, lưu Refresh Token 7 ngày vào CSDL, xoay vòng (Rotation) trong Transaction và thu hồi khi Logout.	PASS
9	DbContext_SaveChangesAsync_AutomaticallyInvalidatesOldCache_OnEntityMutation	Kiểm tra tính năng DbContext tự động xóa sạch các phân vùng cache cũ ngay khi có thao tác Add/Update/Delete dữ liệu.	PASS
10	PasswordHelper_SupportsPbkdf2WithSalt_And_LegacyBackwardCompatibility	Kiểm tra thuật toán băm mật khẩu PBKDF2 kết hợp Salt ngẫu nhiên và hỗ trợ tương thích ngược với chuẩn SHA-256 cũ.	PASS

5.4. Đánh giá kiểm thử chức năng theo yêu cầu đề cương (Mục 22)

Toàn bộ các yêu cầu kiểm thử chức năng quy định tại Mục 22 của đề tài đã được nhóm thực nghiệm và đối chiếu chi tiết:

1. Tài khoản & Phân quyền (Mục 22.1): Đăng nhập thành công với tài khoản đúng; hiển thị lỗi 'Tài khoản không tồn tại' hoặc 'Mật khẩu không chính xác'; tài khoản bị khóa không thể đăng nhập; đăng xuất xóa sạch Session và hủy Cookie xác thực; sinh viên truy cập URL quản trị bị chặn và chuyển về trang AccessDenied.

2. CRUD & Validation (Mục 22.2): Kiểm tra tạo/sửa/xóa học bổng và đơn vị tài trợ; Data Annotation bắt lỗi bỏ trống tên, số suất <= 0, điểm GPA ngoài khoảng [0, 4.0], hạn nộp hồ sơ trước ngày bắt đầu; chặn xóa đơn vị tài trợ hoặc học bổng khi đã có dữ liệu liên quan.

3. Tìm kiếm, Lọc, Sắp xếp & Phân trang (Mục 22.3): Tìm kiếm chính xác theo từ khóa; lọc đơn lẻ và lọc kết hợp đồng thời nhiều tiêu chí; sắp xếp 6 chế độ; phân trang Server-side đúng số lượng trang và duy trì nguyên vẹn các tham số lọc khi chuyển trang.

4. Nộp & Xử lý hồ sơ (Mục 22.4): Chặn nộp hồ sơ khi học bổng đã hết hạn hoặc tạm dừng; chặn sinh viên nộp 2 hồ sơ đang xử lý cho cùng 1 học bổng; chặn nộp hồ sơ khi điểm GPA không đạt; sinh viên hủy hồ sơ thành công khi ở trạng thái Chờ duyệt; cán bộ xét duyệt sơ bộ hồ sơ Chờ duyệt chuyển sang Đạt hoặc Không đạt kèm nhận xét.

5. Lịch phỏng vấn (Mục 22.5): Chỉ cho phép hồ sơ 'Đạt xét duyệt sơ bộ' lập lịch; báo lỗi khi thời gian kết thúc <= thời gian bắt đầu; thuật toán LINQ phát hiện và chặn lập lịch khi sinh viên hoặc hội đồng bị trùng khoảng thời gian phỏng vấn; xác nhận hoàn thành lịch chuyển trạng thái hồ sơ sang 'Đã phỏng vấn'.

6. Kết quả xét học bổng (Mục 22.6): Nhập điểm phỏng vấn (0-100) và công bố kết quả; đếm số hồ sơ đã cấp và hiển thị cảnh báo khi số lượng đạt tối đa số suất học bổng cam kết.

7. Bảo mật quyền dữ liệu sinh viên (Mục 22.7): Sinh viên chỉ xem hồ sơ và thông tin cá nhân của chính mình; sửa mã ID trên URL lập tức bị Controller chặn bằng HTTP 403 Forbid().

8. Dashboard & Thống kê LINQ (Mục 22.8): Số liệu trên Dashboard và 10 chỉ tiêu thống kê LINQ phản ánh chính xác số liệu thực tế trong CSDL SQL Server; tự động cập nhật khi có hồ sơ mới hoặc cập nhật trạng thái.

5.5. Đánh giá ưu điểm, hạn chế thực tế trong mã nguồn và giải pháp khắc phục

Đánh giá khách quan: Nhóm đã tiến hành rà soát kỹ lưỡng toàn bộ mã nguồn trên nhánh Module_5 và ghi nhận các ưu điểm vượt trội cùng một số hạn chế khách quan:

Ưu điểm nổi bật: Hệ thống đáp ứng 100% các chức năng bắt buộc của 05 Module trong đề tài; biên dịch thành công 0 Warning, 0 Error; vượt qua 10/10 bài kiểm thử tự động xUnit; sở hữu các kỹ thuật kiến trúc nâng cao theo chuẩn ứng

dụng doanh nghiệp (JWT + Refresh Token Rotation trong Transaction, Hybrid Redis Cache, Distributed Rate Limiting, Brevo Email Queue, Supabase Cloud Storage, SignalR Realtime).

Hạn chế và điểm cần lưu ý trong mã nguồn: Bên cạnh các ưu điểm trên, việc đối chiếu mã nguồn thực tế với đề tài chỉ ra một số điểm lưu ý kỹ thuật cần hoàn thiện:

1. Kiểm tra thời gian phỏng vấn trong quá khứ: Trong phương thức POST: LichPhongVan/Create, mã nguồn hiện mới kiểm tra 'lich.ThoiGianKetThuc <= lich.ThoiGianBatDau' và kiểm tra trùng lịch bằng LINQ, nhưng chưa bổ sung câu lệnh 'if (lich.ThoiGianBatDau < DateTime.Now)' để chặn thời gian phỏng vấn nằm trong quá khứ tại thời điểm lập lịch theo đúng mô tả tại Mục 9.2 của đề tài.

2. Kiểm soát trạng thái khi nhập kết quả: Tại action GET: KetQuaHocBong/NhapKetQua có kiểm tra chặn hồ sơ Chờ duyệt, tuy nhiên tại action POST: KetQuaHocBong/NhapKetQua chưa kiểm tra chặt chẽ điều kiện 'Chỉ hồ sơ Đã phỏng vấn mới được ghi nhận kết quả cuối cùng' theo đúng nguyên văn Mục 9.4. Ngoài ra, giao diện nhập kết quả có hiển thị cảnh báo hết suất nhưng Controller chưa chặn lưu nếu cán bộ cố tình cấp vượt số suất.

3. Dữ liệu Dashboard quản trị: Tại HomeController.cs, action Index mặc định gán vai trò 'Admin' khi người dùng chưa đăng nhập thay vì chuyển hướng bắt buộc sang trang đăng nhập; dữ liệu thống kê hồ sơ theo tháng (hoSoTheoThang) trên Dashboard quản trị đang sử dụng dữ liệu tĩnh phục vụ giao diện thay vì truy vấn LINQ GroupBy động theo năm/tháng như trong ThongKeController.

Giải pháp khắc phục: Giải pháp khắc phục rất đơn giản: Bổ sung 2 dòng kiểm tra điều kiện 'lich.ThoiGianBatDau < DateTime.Now' tại LichPhongVanController; bổ sung điều kiện 'if (hoSo.TrangThai != "Đã phỏng vấn")' tại KetQuaHocBongController; và bổ sung thuộc tính [RoleAuthorize] tại HomeController.Index. Toàn bộ các giải pháp này hoàn toàn khả thi và nhóm đã nắm vững để trình bày, giải thích trực tiếp trước Hội đồng chấm bảo vệ.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

[TOP Quay lại Menu điều hướng]

1. Kết luận

Sau thời gian nghiên cứu nghiêm túc, phối hợp làm việc nhóm hiệu quả qua GitHub và vận dụng toàn diện các kiến thức của học phần Thực hành Lập trình .NET, Nhóm 02 lớp TI17A3HN đã hoàn thành trọn vẹn Đề tài 22: Xây dựng Hệ thống Quản lý Học bổng và Hồ sơ Xét Học bổng (QuanLyHocBong_UNETI2_TI17A3HN) trên nhánh chính thức Module_5.

Kết quả chính đạt được: Hệ thống đáp ứng xuất sắc các mục tiêu đề ra:

Tính đầy đủ và đúng hạn: Hoàn thành 100% các chức năng bắt buộc của 05 Module nghiệp vụ; mã nguồn tuân thủ nghiêm ngặt mô hình MVC và các quy định về phiên bản (.NET 10 SDK, Entity Framework Core 10.0.12, SQL Server).

Tính đúng đắn của nghiệp vụ: Kiểm soát chặt chẽ luồng vòng đời hồ sơ qua Máy trạng thái hữu hạn (State Machine) gồm 8 trạng thái; ngăn chặn tuyệt đối các thao tác cập nhật sai quy trình.

Chất lượng thuật toán và xử lý dữ liệu: Vận dụng tối đa sức mạnh của ngôn ngữ LINQ trong việc kết hợp Tìm kiếm - Lọc đa tiêu chí - Sắp xếp - Phân trang Server-side; thuật toán kiểm tra chống trùng lịch phỏng vấn hai chiều; và tính toán 10 chỉ tiêu thống kê phức tạp.

Kiến trúc phần mềm nâng cao: Chủ động tích hợp các công nghệ Backend hiện đại theo tiêu chuẩn doanh nghiệp (JWT + Refresh Token Rotation trong Transaction, Hybrid Redis Cache tự động làm mới, Distributed Rate Limiting, Background Email Queue qua Brevo API, Supabase Cloud Storage, SignalR Realtime).

Độ tin cậy và kiểm thử tự động: Xây dựng bộ kiểm thử tự động xUnit với 10/10 Test Cases Passed 100%, bảo đảm độ tin cậy và khả năng bảo trì lâu dài của hệ thống.

2. Hướng phát triển trong tương lai

Hướng mở rộng: Để hệ thống có thể triển khai áp dụng thực tế trên quy mô toàn trường tại UNETI, nhóm đề xuất các hướng nâng cấp và mở rộng trong tương lai:

1. Ký số điện tử: Tích hợp chữ ký số điện tử (Digital Signature) và chứng thư số trên tệp quyết định cấp học bổng PDF xuất tự động cho sinh viên.

2. Tự động hóa giải ngân học bổng: Kết nối trực tiếp với Cổng thanh toán hoặc Hệ thống Core Banking của các ngân hàng đối tác (như Vietcombank) để tự động hóa khâu giải ngân tiền học bổng vào tài khoản của sinh viên trúng tuyển.

3. Ứng dụng Trí tuệ nhân tạo (AI): Tích hợp mô hình học máy (Machine Learning) để phân tích hồ sơ năng lực của sinh viên (ngành học, điểm GPA, giải thưởng) và tự động gợi ý các chương trình học bổng phù hợp nhất (Scholarship Recommendation System).

4. Ứng dụng di động .NET MAUI: Phát triển ứng dụng di động (Mobile App) trên nền tảng .NET MAUI để sinh viên có thể nhận thông báo đẩy tức thì và quản lý hồ sơ thuận tiện trên điện thoại thông minh.

TÀI LIỆU THAM KHẢO

[Quay lại Menu điều hướng]

[1] Khoa Công nghệ Thông tin - Trường Đại học Kinh tế - Kỹ thuật Công nghiệp (UNETI), Đề cương chi tiết học phần và Tài liệu hướng dẫn Thực hành Lập trình .NET, Hà Nội, 2026.

[2] Khoa Công nghệ Thông tin - UNETI, Đề tài 22: Xây dựng hệ thống quản lý học bổng và hồ sơ xét học bổng (ASP.NET Core 10 MVC), Đề tài bài tập lớn môn Thực hành Lập trình .NET, Hà Nội, 2026.

[3] Microsoft Corporation, ASP.NET Core 10.0 Documentation - Overview of ASP.NET Core MVC, https://learn.microsoft.com/en-us/aspnet/core/mvc/overview, truy cập năm 2026.

[4] Microsoft Corporation, Entity Framework Core 10.0 Documentation - Code First Migrations and Relationships, https://learn.microsoft.com/en-us/ef/core/, truy cập năm 2026.

[5] Microsoft Corporation, Language Integrated Query (LINQ) in C# - Query Syntax and Method Syntax, https://learn.microsoft.com/en-us/dotnet/csharp/linq/, truy cập năm 2026.

[6] M. Jones, J. Bradley, N. Sakimura, RFC 7519: JSON Web Token (JWT), Internet Engineering Task Force (IETF), Standards Track, 2015.

[7] Marc Gravell et al., StackExchange.Redis Documentation - High Performance Redis Client for .NET, https://stackexchange.github.io/StackExchange.Redis/, truy cập năm 2026.

[8] Brevo (Sendinblue), Transactional Email API v3 Documentation, https://developers.brevo.com/, truy cập năm 2026.

[9] Supabase Inc., Supabase Storage API Reference - Cloud Object Storage for Modern Applications, https://supabase.com/docs/guides/storage, truy cập năm 2026.

PHỤ LỤC A: BẢNG TỔNG HỢP TẬP TIN MÃ NGUỒN THEO 05 MODULE

[Quay lại Menu điều hướng]

Tuân thủ nghiêm ngặt quy định tại Mục 18 và Mục 23 của đề cương ('Bảng tổng hợp file/chức năng do từng sinh viên phụ trách'), Bảng Phụ lục A.1 liệt kê đầy đủ toàn bộ các tập tin mã nguồn chính trong Solution gắn liền với trách nhiệm phụ trách của từng sinh viên:

Module	Thành viên phụ trách	Tập tin Controller & Dịch vụ (Services)	Tập tin Model, ViewModel & Giao diện (Views)
Module 1	Nguyễn Tuấn Trung (23103100xxx)	• Controllers/TaiKhoanController.cs • Controllers/DonViTaiTroController.cs • Services/Auth/TokenAuthService.cs • Middleware/JwtCookieAuthenticationMiddleware.cs • Attributes/RoleAuthorizeAttribute.cs • Helpers/PasswordHelper.cs	• Models/Entities/TaiKhoan.cs • Models/Entities/RefreshToken.cs • Models/Entities/DonViTaiTro.cs • Models/ViewModels/LoginViewModel.cs • Views/TaiKhoan/* (DangNhap, DoiMatKhau, Index, Create, Edit, AccessDenied) • Views/DonViTaiTro/* (Index, Details, Create, Edit, Delete)
Module 2	Nguyễn Văn Tình (23103100142)	• Controllers/HocBongController.cs • Services/Caching/RedisCacheService.cs • Helpers/CacheKeys.cs	• Models/Entities/HocBong.cs • Views/HocBong/Index.cshtml • Views/HocBong/Details.cshtml • Views/HocBong/Create.cshtml • Views/HocBong/Edit.cshtml • Views/HocBong/Delete.cshtml

Module	Thành viên phụ trách	Tập tin Controller & Dịch vụ (Services)	Tập tin Model, ViewModel & Giao diện (Views)
Module 3	Nguyễn Phi Hùng (23103100148)	• Controllers/SinhVienController.cs • Services/Storage/SupabaseStorageService.cs	• Models/Entities/SinhVien.cs • Models/ViewModels/SinhVienCreateViewModel.cs • Models/ViewModels/SinhVienEditViewModel.cs • Views/SinhVien/* (Index, Details, Create, Edit, ThongTinCaNhan) • Views/HoSoHocBong/NopHoSo.cshtml • Views/HoSoHocBong/HoSoCuaToi.cshtml
Module 4	Phan Sỹ Tú (23103100145)	• Controllers/HoSoHocBongController.cs • Services/Email/RedisBackgroundEmailQueue.cs • Services/Email/EmailQueueHostedService.cs • Services/Email/EmailService.cs • Helpers/EmailTemplateBuilder.cs	• Models/Entities/HoSoHocBong.cs • Views/HoSoHocBong/Index.cshtml • Views/HoSoHocBong/Details.cshtml • Views/HoSoHocBong/XetDuyet.cshtml

Module	Thành viên phụ trách	Tập tin Controller & Dịch vụ (Services)	Tập tin Model, ViewModel & Giao diện (Views)
Module 5	Vũ Mạnh Quỳnh (23103100xxx - Trưởng nhóm)	• Controllers/LichPhongVanController.cs • Controllers/KetQuaHocBongController.cs • Controllers/HomeController.cs • Controllers/ThongKeController.cs • Controllers/ThongBaoController.cs • Hubs/NotificationHub.cs • Services/Notifications/NotificationService.cs • Services/RateLimiting/RedisRateLimitService.cs • Middleware/DistributedRateLimitingMiddleware.cs • tests/QuanLyHocBong.Tests/InfrastructureTests.cs	• Models/Entities/LichPhongVan.cs • Models/Entities/KetQuaHocBong.cs • Models/Entities/ThongBao.cs • Models/ViewModels/DashboardViewModel.cs • Models/ViewModels/StudentDashboardViewModel.cs • Models/ViewModels/ThongKeViewModel.cs • Views/LichPhongVan/* (Index, Create) • Views/KetQuaHocBong/* (Index, NhapKetQua) • Views/Home/* (Index, StudentDashboard) • Views/ThongKe/Index.cshtml

PHỤ LỤC B: BẢNG PHÂN CÔNG NHIỆM VỤ VÀ ĐÁNH GIÁ THÀNH VIÊN

[Quay lại Menu điều hướng]

Bảng Phụ lục B.1 tổng kết khối lượng công việc thực tế, vai trò đảm nhận và tỷ lệ đóng góp của từng thành viên trong Nhóm 02 thực hiện Đề tài 22:

STT	Họ và tên	Mã sinh viên	Module phụ trách	Nội dung công việc hoàn thành chi tiết	Đánh giá mức độ
1	Nguyễn Trung Tuấn	23103100xxx	Module 1	• Xây dựng Entity TaiKhoan, DonViTaiTro, RefreshToken. • Lập trình xác thực đăng nhập, kiểm tra mật khẩu PBKDF2/SHA-256. • Thiết lập Session và phân quyền RoleAuthorizeAttribute trên Controller. • Hạ tầng xác thực kép JWT 15 phút & Refresh Token 7 ngày trong HttpOnly Cookie. • Lập trình CRUD Đơn vị tài trợ, kiểm tra ràng buộc không cho xóa khi có học bổng liên kết.	100% Hoàn thành xuất sắc
2	Nguyễn Văn Tình	23103100142	Module 2	• Xây dựng Entity HocBong kèm Data Annotations validation. • Lập trình chuỗi truy vấn IQueryable kết hợp đồng thời Search + Filter + Sort + Pagination. • Thiết kế giao diện danh sách học bổng, chi tiết, thêm, sửa, xóa học bổng. • Xử lý duy trì từ khóa tìm kiếm, bộ lọc và thứ tự sắp xếp khi phân trang. • Kiểm soát nghiệp vụ chặn xóa học bổng khi đã có hồ sơ đăng ký.	100% Hoàn thành xuất sắc

STT	Họ và tên	Mã sinh viên	Module phụ trách	Nội dung công việc hoàn thành chi tiết	Đánh giá mức độ
3	Nguyễn Phi Hùng	23103100148	Module 3	• Xây dựng Entity SinhVien và liên kết 1-1 với TaiKhoan. • Xây dựng giao diện hồ sơ cá nhân và cập nhật ảnh đại diện lên Supabase Storage. • Kiểm tra 03 điều kiện nộp hồ sơ bằng LINQ (học bổng mở, trong hạn, không trùng, GPA đạt chuẩn). • Lập trình chức năng hủy hồ sơ hợp lệ cho sinh viên. • Bảo mật quyền dữ liệu: Ngăn chặn sinh viên can thiệp ID trên URL để xem hồ sơ người khác.	100% Hoàn thành xuất sắc
4	Phan Sỹ Tú	23103100145	Module 4	• Xây dựng giao diện danh sách tiếp nhận hồ sơ ứng tuyển cho cán bộ (Search, Filter, Sort). • Xây dựng chức năng xét duyệt sơ bộ chuyển trạng thái từ Chờ duyệt sang Đạt / Không đạt sơ bộ. • Tự động ghi nhận NgayXuLy và NhanXetXetDuyet. • Quản lý máy trạng thái Finite State Machine nghiêm ngặt. • Kiểm soát số suất học bổng bằng LINQ CountAsync và xuất danh sách hồ sơ ra file CSV.	100% Hoàn thành xuất sắc

STT	Họ và tên	Mã sinh viên	Module phụ trách	Nội dung công việc hoàn thành chi tiết	Đánh giá mức độ
5	Vũ Mạnh Quỳnh (Trưởng nhóm)	23103100xxx	Module 5 & Tổng thể	• Thiết kế và lập trình thuật toán LINQ kiểm tra chống trùng lịch phỏng vấn hai chiều cho Sinh viên và Hội đồng. • Lập trình cập nhật kết quả phỏng vấn, kiểm soát số suất và cảnh báo hết suất. • Phát triển Dashboard quản trị tổng hợp 9 chỉ số và Dashboard cá nhân hóa cho sinh viên. • Lập trình 10 chỉ tiêu thống kê chuyên sâu bằng LINQ và xuất báo cáo CSV. • Tích hợp toàn diện các hạ tầng Backend nâng cao: Hybrid Redis Cache, Rate Limiting, Brevo Email Queue, SignalR. • Xây dựng bộ kiểm thử tự động xUnit (10/10 Test Passed); quản trị Repository GitHub và viết báo cáo tổng kết.	100% Hoàn thành xuất sắc